

FTIAS Manual

FLIGHT TEST INTERACTIVE ANALYSIS SUIT

V-00

ABSTRACT

The Flight Test Interactive Analysis Suite (FTIAS) is a web-based engineering support system designed to enable structured, traceable, and reproducible analysis of flight-test data. The system integrates data ingestion, parameter exploration, deterministic engineering calculations, retrieval-augmented analysis, and mission-risk assessment workflows within a unified architecture.

FTIAS is built on a deterministic-first design philosophy, where explicit engineering logic is prioritized for quantitative results, and AI-assisted components provide contextual interpretation, documentation support, and narrative explanation. The system enforces dataset versioning, analysis immutability, and provenance tracking to ensure that all outputs can be traced back to their originating data and processing context.

The platform supports multiple analysis domains, including takeoff and landing performance, handling and control response, air-data and performance trends, vibration and loads screening, and flutter-support pre-screening. Each analysis mode operates within defined applicability boundaries and communicates result quality through structured control fields such as confidence, coverage, and warning level.

In addition to analytical capabilities, FTIAS includes a Flight Risk Assessment Tool (FRAT) workflow to support structured mission-risk evaluation, scoring, and decision tracking. All outputs, including analysis reports and FRAT assessments, are generated from preserved artifacts to maintain consistency and auditability.

This manual provides a complete description of the system architecture, data model, operational workflows, analysis methodologies, limitations, and user responsibilities required for effective and responsible use of FTIAS.

EXECUTIVE SUMMARY

The Flight Test Interactive Analysis Suite (FTIAS) is designed to support engineers in analyzing, interpreting, and documenting flight-test data through a structured, traceable workflow.

The system combines multiple capabilities into a single environment:

- ingestion of large flight-test datasets with version control
- interactive parameter exploration and charting
- deterministic engineering analysis for key domains
- AI-assisted interpretation and standards support
- structured analysis outputs with confidence and applicability controls
- reproducible report generation
- mission-risk assessment through an integrated FRAT workflow

A key principle of FTIAS is traceability. Every analysis result is linked to a specific dataset version, processing context, and saved artifact. This ensures that results remain consistent over time and can be reviewed, validated, and reproduced.

Another fundamental principle is the separation between deterministic engineering logic and AI-generated narrative. Deterministic modules provide bounded quantitative results based on available telemetry data, while AI components support interpretation, documentation, and contextual explanation. This approach reduces ambiguity and prevents unsupported conclusions.

The system is intentionally designed with bounded capabilities. Each analysis mode operates within defined limits, and results are accompanied by structured control indicators such as confidence, coverage, applicability, and warning level. These indicators guide the user in understanding how strongly a result can be interpreted.

FTIAS is not a certification or approval tool. It is an engineering support platform intended for:

- exploratory data analysis
- trend identification
- preliminary performance assessment
- technical reporting
- risk-awareness support

Final engineering decisions, certification conclusions, and operational approvals remain the responsibility of qualified personnel.

This manual provides the necessary technical and operational guidance to use FTIAS effectively, understand its outputs, and apply its results within appropriate engineering boundaries.

*"The following chapters describe the system architecture, workflows,
and usage in detail."*

Contents

CHAPTER 1 - Introduction, Architecture, and End-to-End Logic	15
Overview	16
Simple block diagram.....	16
The system in plain language	17
Frontend	17
Backend	17
Database.....	17
Docker	17
End-to-end algorithm of the whole application	17
Main workflows, from simple to detailed	17
Flight test creation and ownership.....	17
CSV upload and ingestion	18
Dataset versioning.....	18
Parameter exploration and charting.....	18
AI Standards Query and AI analysis	18
Report / PDF export	19
Deletion and data integrity.....	19
Core backend entities and their relationships.....	19
Simple relationship chain example:	20
Correct connections between frontend, backend, database, and Docker	20
Frontend to backend	20
Backend to database.....	20
Backend to AI/document services	20
Docker to services	21
Browser to Docker	21
Practical operations model	21
Restart vs rebuild	21
Migrations	21
Why failures often look like 'Failed to fetch.'	21
Why Docker matters operationally.....	21
Algorithm viewed in layers.....	21
Layer 1 - user algorithm	21
Layer 2 - frontend algorithm	21

Layer 3 - backend algorithm.....	21
Layer 4 - data algorithm	22
Layer 5 - runtime algorithm	22
A progressively deeper mental model	22
Level A - simple	22
Level B - useful	22
Level C - engineering correct.....	22
Level D - maintainable	22
Level E - production-minded	22
Final summary	22
CHAPTER 2 - System Overview	23
System Philosophy and Design Principles	23
What FTIAS does.....	23
Major capabilities	24
Flight-test data management	24
Dataset versioning and provenance	25
Parameter exploration and charting.....	25
AI Standards Query	25
AI Analysis	26
Deterministic engineering calculations	26
Retrieval-augmented reference support	27
Prompt-to-mode guard.....	28
Confidence, coverage, and applicability controls.....	28
Report and PDF generation	29
FRAT/mission risk workflow.....	29
Engineering boundaries	30
Deterministic does not mean unlimited	30
RAG support is advisory.....	30
Some modes are screening only.	30
Certification corrections must be explicit.	31
User responsibility	31
System overview summary	32
CHAPTER 3 - Architecture and Component Connections	33
Frontend	33
Backend.....	34

Database	35
Docker	36
External AI/Document Services	36
Main Runtime Services	38
Component Responsibilities	39
Architecture by Workflow	39
Upload workflow	39
Parameter workflow	39
AI analysis workflow	40
Report workflow	40
FRAT workflow	40
Why This Architecture Matters	40
Chapter summary	41
CHAPTER 4 - Installation and Docker Environment	42
Prerequisites	42
Repo Structure	42
Docker Services	43
Starting the Application	44
Stopping the Application	44
Restart vs Rebuild	45
Restart	45
Rebuild	45
Migrations	46
Applying a migration	46
Migration rule	46
Logs	47
Backend logs	47
Frontend logs	47
PostgreSQL logs	47
Environment Variables	48
Common Runtime Problems	48
"Failed to fetch"	48
Missing database column	49
Foreign-key violation	49
Frontend does not show new UI changes	49

The backend does not reflect the new logic	50
Development Validation Commands	50
Recommended Operational Sequence After New Code	50
Safe Handling of Data	51
Chapter Summary	51
CHAPTER 5 - Frontend Operation.....	53
Pages	53
Dashboard/Flight Test Detail	53
Upload Data.....	54
Parameters.....	54
Document Library	55
AI Standards Query	55
FRAT/Mission Risk Workflow	56
Navigation.....	56
Dataset Selection	57
AI Mode Selection.....	57
Takeoff Performance	58
Landing Performance	58
Performance/Climb/ Air Data.....	58
Handling/Control Response	59
Vibration & Loads	59
Flutter Support Pre-screen	60
General Summary.....	60
Prompt-to-Mode Warnings	61
Saved Analysis Reopen	61
Report Export from the Frontend	62
FRAT Page.....	62
User Checklist for Frontend Operation	63
Common Frontend Issues	64
Page shows "Failed to fetch"	64
New UI changes are not visible.....	64
The dataset label looks wrong.	65
Saved parameter sets disappear.	65
Chart export does not work.....	65
Frontend Operation Summary	65

CHAPTER 6 - Backend Operation	67
Routers	67
Services	68
Deterministic Modules	69
Saved Artifact Model	70
Deletion Integrity	72
Routing and Controls	73
Capability Catalog	74
Analysis Controls	75
Document Retrieval and RAG Support	75
Report Generation	76
FRAT Backend Workflow	77
Backend Operation Summary	77
CHAPTER 7 - Database and Data Model	79
Main Entities	79
User	80
FlightTest	80
DatasetVersion	81
IngestionSession	81
TestParameter	82
DataPoint	83
AnalysisJob	83
Document	84
DocumentChunk	85
FRATAssessment	85
Key Relationships	86
Flight Test to Dataset Version	87
Dataset Version to Data Points	87
Ingestion Session to Dataset Version	88
Flight Test to Analysis Job	88
Flight Test to FRAT Assessment	88
Document to Document Chunk	89
Provenance Model	89
Dataset Provenance	89
Analysis Provenance	90

Report Provenance.....	91
FRAT Provenance.....	91
Immutability Model.....	92
Dataset Immutability	92
Analysis Job Immutability	92
FRAT Immutability	93
Migration Discipline	93
Safe Migration Practice	94
Foreign Keys and Deletion Order	94
Database and Data Model Summary.....	95
CHAPTER 8 - Data Ingestion Workflow	97
Flight Test Creation	97
CSV Upload	98
Ingestion Sessions.....	99
Dataset Version Creation	100
Parameter Detection	100
Timestamp Handling	101
Data Point Insertion.....	102
Validation.....	102
Failure Cases	103
Upload History	104
Active Dataset After Upload	105
Large Dataset Considerations.....	105
Ingestion and Provenance.....	106
User Checklist for Data Ingestion.....	107
Data Ingestion Workflow Summary	107
CHAPTER 9 - Dataset Versioning and Provenance	109
Dataset Versioning Concept	109
Active Dataset.....	109
Dataset Selection in Workflows.....	109
Dataset Provenance	110
Provenance Chain	111
Why Versioning Matters.....	111
Dataset Versioning Summary	111
CHAPTER 10 - AI Analysis Workflow.....	112

Analysis Workflow Overview.....	112
Analysis Modes	112
Deterministic vs AI Components	113
Prompt-to-Mode Guard	113
Capability Routing	114
Analysis Controls.....	114
RAG (Retrieval-Augmented Generation).....	115
Saved Analysis Job	115
Analysis Limitations	116
AI Analysis Summary.....	116
CHAPTER 11 - Charts, Parameters, and Comparison Features.....	117
Parameter Explorer.....	117
Parameter Selection.....	117
Favourites	117
Saved Parameter Sets.....	118
Charting.....	118
Overlays and Markers	118
Dataset Comparison	119
Chart Export.....	119
Role of Charts in Analysis	119
Common Issues	120
Missing parameters.....	120
Incorrect charts.....	120
Performance issues	120
Best Practices	120
Charts and Parameters Summary	120
CHAPTER 12 - Report and PDF Generation	122
Purpose of Report Generation.....	122
Immutable Artifact Principle.....	122
Report Generation Workflow	123
Report Structure.....	123
Controls in Reports	123
Warning Escalation	124
Provenance in Reports	124
Limitations of Reports	124

Report Generation Summary	125
CHAPTER 13 - FRAT / Mission Risk Workflow (Detailed)	126
Purpose of FRAT	126
FRAT Workflow Overview	126
Risk Scoring	126
Hard-Stop Logic	127
Lifecycle States	127
User Checklist for FRAT	128
Interpretation of FRAT Results	128
FRAT and Analysis Integration	128
FRAT Summary	129
CHAPTER 14 - Analysis Interpretation Guide	130
Reading an Analysis Result	130
Understanding Control Fields	130
Data Quality Impact	130
Mode Limitations	131
When to Trust Results	131
When to Be Cautious	131
When to Escalate	131
Interpretation Summary	132
CHAPTER 15 - System Limitations and User Responsibility	133
System Limitations	133
Not a Certification Tool	133
User Responsibility	133
Responsible Usage	133
Responsibility Summary	134
CHAPTER 16 - Troubleshooting	135
Failed to Fetch	135
Migration Errors	135
Dataset Issues	135
Analysis Issues	135
Frontend Issues	135
Troubleshooting Summary	136
CHAPTER 17 - References and Technology Links	137
Application Architecture	137

Frontend/Web Application	137
React	137
TypeScript	137
Vite	137
pnpm	137
Backend/API Layer	137
Python	137
FastAPI	138
Pydantic	138
SQLAlchemy	138
Pytest	138
Database and Persistence	138
PostgreSQL	138
Database migrations	138
Runtime and Deployment	139
Docker	139
Docker Compose	139
AI, Retrieval, and Reporting Concepts	139
Retrieval-Augmented Generation (RAG)	139
OpenAI API / AI model provider	139
PDF reporting	139
Engineering and Flight-Test Reference Areas	139
CHAPTER 18 - Definitions and Key Concepts	141
Active Dataset	141
AI Analysis	141
AI Standards Query	141
Analysis Job	141
Analysis Mode	141
Applicability Boundary	141
Backend	141
Capability Catalog	141
Confidence/Coverage/Applicability Controls	141
DataPoint	142
Dataset Version	142

Deterministic Calculation	142
Docker	142
Docker Compose	142
Document Chunk.....	142
Document Library	142
External AI Service	142
Flight Test	142
Foreign Key	142
FRAT	142
Frontend	143
Hard-Stop	143
Immutable Artifact.....	143
Ingestion Session	143
Migration.....	143
Mode-Aware Retrieval	143
Parameter Explorer.....	143
Prompt-to-Mode Guard.....	143
Provenance.....	143
RAG.....	143
Report Export	143
Result Strength	144
Screening	144
Telemetry.....	144
Test Parameter	144
Traceability.....	144
WOW	144
ANNEX A - Operational Checklists.....	145
ANNEX A.1 — Data Upload Checklist	145
ANNEX A.2 — AI Analysis Checklist	145
ANNEX A.3 — FRAT Checklist	146
ANNEX A.4 — Dataset Version Control Checklist.....	146
ANNEX A.5 — Troubleshooting Quick Checklist.....	146
ANNEX B - FRAT Input Interpretation Guide	148
B.1 General Scoring Philosophy	148
Hard stops always override numerical scoring	148

B.2 Assessment Inputs.....	148
Assessment Name	148
Dataset Version	149
Requested Authority	149
Manual Adjustment (-10 to +10)	149
Linked Analysis Job	150
B.3 Risk Categories (0–20 each).....	150
The FRAT scoring table	151
Mission/Test Profile	151
Weather / Environment	151
Runway / Operational.....	152
Aircraft System Status	152
Crew Readiness.....	152
B.4 Hard-Stop Flags.....	153
Critical System Unavailable	153
Mandatory Data Missing.....	153
Crew Unfit	153
Hard-Stop Principle	153
B.5 Notes Sections	153
Review Notes	153
Override / Rationale Notes	154
Transition Notes	154
B.6 Final Score Interpretation	154
B.7 Total FRAT Score Composition	154
B.7 Key Principles.....	155
B.8 Example	155
Low Risk, Go.....	155
Hard Stop, No-Go	156
Abbreviations	158

CHAPTER 1 - Introduction, Architecture, and End-to-End Logic

A gradual explanation from a simple view to full system behavior for engineering, product, and maintenance understanding

Project	Flight Test Interactive Analysis Suite (FTIAS)
Audience	Project owner/engineering review
Focus	Frontend-backend-database-Docker relationships, algorithm, and workflows
Prepared for	User architecture briefing

This document explains the current FTIAS structure from a simple mental model to the internal workflow details.

It is intentionally written in layers so a reader can stop at the level of detail they need.

Overview

FTIAS is a flight-test data and engineering knowledge application. At the highest level, it lets an engineer upload flight-test data, organize it into flight tests and dataset versions, visualize parameters, compare datasets, query technical documents with AI assistance, run saved AI analyses, and export professional reports.

The easiest way to understand the system is to see it as three main roles running together inside Docker:

- Frontend: the visible application used by the engineer.
- Backend: the decision-making and data-control layer.
- Docker: the runtime shell that starts and connects the services consistently.

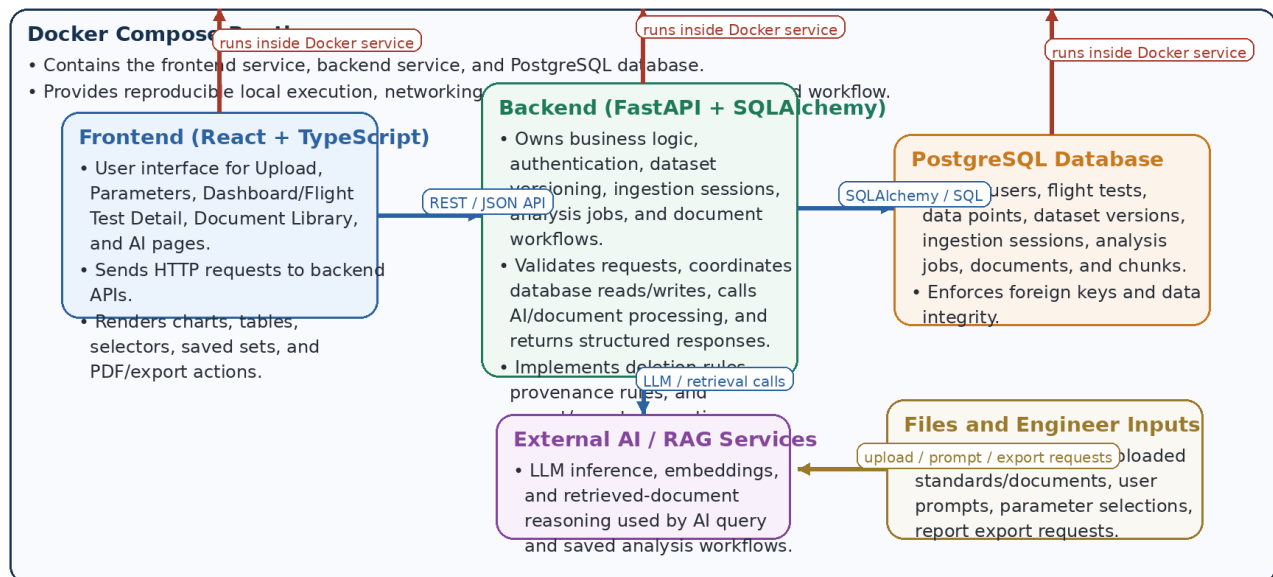
The database is the project's persistent memory, while the AI and document services are external capabilities that the backend uses as needed.

Simple block diagram

The diagram below shows the correct high-level connection pattern. The frontend does not talk directly to the database. The backend is the control point between user actions, stored data, AI services, and export/report generation.

FTIAS - Simple Architecture and Connection Map

Frontend, Backend, Docker, Database, and external AI/document services



Correct high-level connection: Browser -> Frontend -> Backend -> Database / AI services. Docker hosts and connects the services.

Figure 1. High-level FTIAS architecture and service connections.

The system in plain language

Frontend

The frontend is the interface that the engineer sees in the browser. It contains pages for Upload, Parameters, Dashboard/Flight Test Detail, AI Standards Query, Document Library, and report-related actions. Its main job is presentation, interaction, and sending structured requests to the backend.

Backend

The backend is the operational brain. It validates requests, applies security and user ownership rules, reads and writes data in PostgreSQL, performs ingestion logic, manages dataset versions, saves immutable analysis jobs, and prepares exports. When AI is needed, the backend also prepares prompts and context before calling external services.

Database

PostgreSQL is the durable storage layer. It contains project records, including users, flight tests, dataset versions, ingestion sessions, data points, analysis jobs, documents, and document chunks. Foreign keys protect the relationships between these records.

Docker

Docker Compose is not business logic; it is the runtime container system. It starts with the frontend, backend, and PostgreSQL services and provides a stable way to repeatedly run the same stack. In practical terms, Docker is the environment wrapper that keeps the parts connected correctly.

End-to-end algorithm of the whole application

At the broadest level, almost every FTIAS action follows the same algorithm:

1. The engineer acts as the frontend, such as selecting a flight test, uploading a CSV, changing a dataset version, requesting an AI analysis, or exporting a report.
2. The frontend converts that action into an HTTP request to the backend API.
3. The backend authenticates the user, checks ownership and scope, and validates the request.
4. The backend reads or updates the database, and if needed, calls external AI or document-processing services.
5. The backend returns a structured JSON response or a file response.
6. The frontend renders the result: charts, status banners, history, warnings, saved analysis metadata, or downloads.

Main workflows, from simple to detailed

Flight test creation and ownership

The user creates a flight test from the front end. The backend checks whether the test name already exists for that user, then stores the new flight-test record.

The important rule is user tenancy: a user should only see and modify the flight tests they own. This rule is enforced in the backend queries, not only in the frontend.

CSV upload and ingestion

When a CSV is uploaded, the backend first verifies that the flight test exists and belongs to the current user. It then creates two records before processing the data: an ingestion session and a new dataset version.

The ingestion session records the upload's operational state, such as processing, success, or failure. The dataset version represents the uploaded dataset as an immutable version under that flight test.

The backend parses the CSV structure, validates the timestamp column, creates any missing parameter definitions, and then inserts data points in batches. On success, the dataset version is marked successfully and set as the active dataset for that flight test. On failure, the ingestion session and dataset version are marked failed with an error detail.

Dataset versioning

Dataset versioning is one of the most important architectural ideas in the current project. Instead of silently overwriting old data, each upload becomes a new immutable dataset version, such as v1, v2, and so on.

This gives the project auditability and reproducibility. Parameters, plots, AI analyses, and reports can be tied to a specific dataset version rather than a moving target.

The flight test also has an "active_dataset_version_id." This pointer indicates which dataset version should be treated as the default working dataset when a specific version is not explicitly requested.

Parameter exploration and charting

In the Parameters page, the frontend requests the parameter list and parameter data for a selected flight test and dataset version. The backend responds with the matching dataset-scoped series.

The frontend then renders time-series or correlation views. Recent work added saved parameter sets, favourites, linked hover behavior, threshold overlays, event markers, compare-dataset mode, and improved PNG export.

The important architectural point is that the frontend only renders what the backend returns. Dataset scoping remains backend-controlled, while the frontend manages visual interaction and user workflow memory, such as favorites and saved sets.

AI Standards Query and AI analysis

The project has two major AI-oriented paths. The first is a document-centered query over standards and technical references. The second is flight-test-data-centered analysis tied to a flight test and a dataset version.

For saved flight-test analyses, the backend computes and stores an immutable analysis job. That job includes provenance such as the dataset version used, model metadata, analysis text, retrieved-source snapshots, and snapshot data for parameter statistics.

This immutability is critical. A reopened analysis should not drift when later data changes. The saved analysis job is therefore closer to a frozen engineering artifact than to a transient chat response.

Report / PDF export

When the user exports a saved analysis, the backend uses the persisted analysis job and associated provenance to build the report. The correct design principle is that export should rely on the saved job, not on recomputing live data from the database unless that is explicitly intended.

This creates reproducible reports. The report reflects the chosen flight test, the chosen dataset version, and the saved analysis provenance.

Deletion and data integrity

Deletion is simple at the user level but complex internally because the database contains dependency chains. A flight test may have data points, dataset versions, ingestion sessions, and analysis jobs. The flight test may also point to an active dataset version, and dataset versions may point back to ingestion sessions through `source_session_id`.

For that reason, deletion must follow a safe order. The backend now clears the active dataset pointer, then deletes dependent entities in foreign-key-safe order. This is a good example of how backend logic protects data integrity beyond what the frontend can see.

Core backend entities and their relationships

The easiest way to understand the backend is to see it as a graph of records rather than a list of files. The table below summarizes the main entities.

Entity	Purpose	Key links	Why it matters
User	Owns and authenticates application activity	Flight-test, Document, Ingestion Session, Analysis Job	Defines tenancy and access scope
FlightTest	Top-level engineering test record	<code>created_by_id</code> , <code>active_dataset_version_id</code>	Container for all test-specific data
DatasetVersion	Immutable upload version under a flight test	<code>flight_test_id</code> , <code>source_session_id</code>	Enables auditability and compare-dataset logic
IngestionSession	Operational record of an upload	<code>flight_test_id</code> , <code>dataset_version_id</code>	Captures processing state and failures
DataPoint	Time-series measurement value	<code>flight_test_id</code> , <code>parameter_id</code> , <code>dataset_version_id</code>	Main source for charts and stats

Entity	Purpose	Key links	Why it matters
TestParameter	Parameter definition and unit metadata	Referenced by Datapoint	Allows parameter naming and unit consistency
AnalysisJob	Immutable saved AI analysis artifact	flight_test_id, dataset_version_id, created_by_id	Preserves reproducible AI outputs
Document	Uploaded engineering reference file	uploaded_by_id	RAG source container
DocumentChunk	Searchable chunk of a document	document_id	Used in retrieval and citation support

Simple relationship chain example:

- A user owns a flight test.
- A flight test contains dataset versions.
- A dataset version originates from an ingestion session.
- A dataset version contains data points.
- A saved analysis job points to the flight test and the dataset version used.
- A report export should therefore be able to tell exactly which flight test and which dataset version produced the result.

Correct connections between frontend, backend, database, and Docker

Frontend to backend

The frontend should communicate only through backend APIs. It should never query the database directly. This keeps security, validation, and business logic centralized.

Backend to database

The backend is the only layer that should perform application-level reads and writes against PostgreSQL. It translates application actions into controlled database operations and respects foreign-key order and tenancy rules.

Backend to AI/document services

The backend prepares structured prompts, retrieves document chunks or dataset statistics, and then calls external AI services. This prevents the frontend from having to know model details or data access logic.

Docker to services

Docker Compose starts the frontend, backend, and database containers and defines the network paths between them. For example, the backend can reach Postgres via the Compose service connection, while the browser reaches the frontend, which then calls the backend via the configured host/port routes.

Browser to Docker

The browser does not directly know Docker concepts. It simply accesses the exposed frontend address and indirectly benefits from Docker, which has already started and connected the services behind the scenes.

Practical operations model

Restart vs rebuild

If the code changes but the dependencies and image contents remain the same, a restart is usually enough. If dependencies, Dockerfile contents, or image-level packages change, a rebuild is required.

Migrations

When new database schema migrations are added, they must be applied to PostgreSQL before restarting backend logic that expects the new schema. Otherwise, the front end often shows generic fetch failures caused by backend startup or runtime errors.

Why failures often look like 'Failed to fetch.'

The frontend usually sees only that the API call failed. The real cause is often in the backend logs: a migration mismatch, a foreign-key violation, or a startup failure.

Why Docker matters operationally

Docker makes the runtime reproducible, but it also means that code, migrations, container restart behavior, and bind-mounted files must be understood together rather than as isolated pieces.

Algorithm viewed in layers

Layer 1 - user algorithm

Select a flight test, upload or choose a dataset, inspect the parameters, run the analysis, and export the results.

Layer 2 - frontend algorithm

Collect user state, call APIs, render results, and preserve local workflow settings such as favourites and saved sets.

Layer 3 - backend algorithm

Authenticate, validate, resolve dataset scope, execute database work, enforce integrity, call AI/document services, and produce structured responses.

Layer 4 - data algorithm

Persist state transitions and provenance: uploads become dataset versions, analyses become immutable jobs, reports derive from saved artifacts.

Layer 5 - runtime algorithm

Docker launches the services, networking enables communication among frontend, backend, and database, migrations align the schema with the code, and logs reveal runtime failures.

A progressively deeper mental model

Level A - simple

FTIAS is a web app where engineers upload test data to get plots, AI support, and reports.

Level B - useful

The frontend is the screen, the backend is the logic, the database is the memory, and Docker is the launcher/container environment.

Level C - engineering correct

A flight test is not only a file upload. It is a controlled object with immutable dataset versions, ingestion history, and saved analyses tied to specific provenance.

Level D - maintainable

The most important behaviors must live in the backend: tenancy, validation, dataset scoping, deletion order, saved-job immutability, and report provenance.

Level E - production-minded

The health of the whole system depends on alignment between code, schema migrations, service restarts, and external AI configuration.

Final summary

- FTIAS is best understood as a layered engineering system rather than only as a UI.
- The frontend is responsible for interaction and visualization.
- The backend is responsible for rules, integrity, provenance, and orchestration.
- The database is responsible for persistent relationships and auditability.
- Docker is responsible for packaging and connecting the services into a repeatable runtime.
- The project becomes easier to maintain when each problem is assigned to the correct layer.

CHAPTER 2 - System Overview

FTIAS, the Flight Test Interactive Analysis Suite, is an engineering support application designed to organize flight-test data, visualize parameters, support technical analysis, connect analysis results to reference documents, and generate traceable reports.

The system is not only a plotting tool, but also an AI interface. It is a layered engineering workflow that combines uploaded telemetry, dataset versioning, deterministic calculations, document retrieval, analysis controls, provenance, and report generation.

The main purpose of FTIAS is to help an engineer move from raw flight-test data to structured engineering insight while preserving traceability.

System Philosophy and Design Principles

- deterministic-first
- traceability over convenience
- immutable artifacts
- bounded engineering outputs
- user responsibility

What FTIAS does

FTIAS allows the user to create flight-test records, upload CSV data, inspect parameters, compare dataset versions, run analysis modes, query reference documents, and export engineering reports.

At a practical level, the application supports the following activities:

1. Create and manage flight-test records.
2. Upload flight-test CSV data.
3. Store each successful upload as an immutable dataset version.
4. Select active or historical dataset versions.
5. Explore parameters and plot time-series data.
6. Save favorite parameters and reusable parameter sets.
7. Compare parameters across dataset versions.
8. Run AI-assisted analysis linked to a selected flight test and dataset version.
9. Use deterministic engineering calculators where supported.
10. Use retrieval-augmented reference support from uploaded documents.

11. Save analysis jobs as immutable artifacts.
12. Reopen saved analyses by ID.
13. Export reproducible PDF reports from saved analysis jobs.
14. Run FRAT/mission risk workflows.
15. Preserve provenance across datasets, analyses, and reports.

The key point is that FTIAS is designed around a traceable engineering workflow, not on temporary screen output.

A chart, an AI analysis, or a PDF report should always be understood in relation to:

- the flight test used
 - the dataset version used
 - the selected analysis mode
 - the parameters available
 - the source documents retrieved
 - the deterministic calculations performed
 - the saved analysis job or FRAT record generated
-

Major capabilities

FTIAS currently includes several major capability families.

Flight-test data management

The system organizes uploaded data under flight-test records. A flight test acts as the main container for related datasets, analyses, reports, and review activity.

Each upload becomes a dataset version. This prevents hidden-overwrite behavior and supports comparison across different uploads or processing attempts.

For example, a flight test may contain:

- dataset version v1
- dataset version v2
- dataset version v3

Only one dataset version may be active by default, but historical versions remain available for inspection and comparison.

Dataset versioning and provenance

Dataset versioning is one of the core design principles of FTIAS.

Each successful upload is preserved as an immutable version. Analyses and reports can then refer to a specific version rather than relying on whatever data is active at a later time.

This supports reproducibility. If an engineer exports a report from Analysis Job #42, the report should still reflect the dataset version and analysis artifact used at the time the job was created.

Chapter 1 of the manual already introduces this concept as one of the most important architectural ideas in the current project. Each upload becomes a new immutable dataset version instead of silently overwriting old data.

Parameter exploration and charting

FTIAS provides parameter exploration tools for large flight-test datasets.

The user can:

- search parameters
- mark favourites
- save parameter sets
- plot selected parameters
- use threshold overlays
- use event markers
- compare dataset versions
- export chart images

The charting workflow is intended to support engineering review. It helps the user identify trends, compare signals, inspect anomalies, and prepare analysis questions.

The frontend renders the charts, but the data scope remains controlled by the backend. This means the selected flight test and dataset version determine which parameter data is returned.

AI Standards Query

The AI Standards Query workflow is document-centric.

The user uploads or uses available technical documents, then asks questions about those references. The backend retrieves relevant document chunks and provides an answer with source references.

This workflow is useful for:

- standards interpretation

- technical reference lookup
- finding related sections in uploaded manuals or guidance documents
- supporting analysis narrative with source context

This path is advisory and reference-support oriented. It does not replace engineering judgment.

AI Analysis

The AI Analysis workflow is flight-test-data-centered.

The user selects a flight test, a dataset version, and an analysis mode. The backend then combines:

- dataset statistics
- available telemetry parameters
- deterministic calculators where applicable
- retrieved useful document context
- analysis-mode rules
- applicability and confidence controls

The output is saved as an immutable analysis job. This allows the user to reopen the analysis later and export a report from the saved artifact.

The current analysis modes include:

- Takeoff Performance
- Landing Performance
- Performance / Climb / Air Data support
- Handling / Control Response
- Vibration & Loads
- Flutter Support Pre-screen
- General Summary

Each mode has its own applicability limits. Some modes are stronger deterministic workflows, while others remain bounded screening or advisory workflows.

Deterministic engineering calculations

FTIAS includes deterministic calculators for supported engineering workflows.

Current deterministic or bounded deterministic areas include:

- takeoff ground-roll estimate
- landing rollout estimate
- general performance trend metrics
- atmosphere / air-data support
- handling / control-response summaries
- buffet/vibration screening
- flutter-support pre-screening

These deterministic calculations are intended to be more authoritative than free-form AI text within their defined boundaries.

However, deterministic does not automatically mean certification-ready. Each result must still be interpreted according to its limitations, assumptions, and applicability boundaries.

For example:

- A takeoff result may estimate ground roll to liftoff
- but it is not a certification-corrected takeoff distance unless the required correction models are explicitly applied

Retrieval-augmented reference support

FTIAS uses uploaded documents as reference material for standards and technical context.

The retrieval layer can use:

- document chunks
- source metadata
- authority type
- domain tags
- capability tags
- analysis mode

This helps the system retrieve more relevant sources for the selected analysis mode.

For example:

- Takeoff analysis should prefer takeoff/performance references
- Handling analysis should prefer handling qualities and control-response references
- Flutter-support analysis should prefer flutter, vibration, aeroelastic, or risk-related references where available

The retrieved sources support the narrative and cross-check sections, but they do not replace deterministic computation.

Prompt-to-mode guard

FTIAS includes a prompt-to-mode guard to reduce the risk of using the wrong analysis mode.

For example, if the user selects **Takeoff Performance** but writes a prompt about aileron deflection and stick position, the system can detect that the prompt is more related to handling qualities than takeoff performance.

The guard can then display a warning, show the detected intent, and suggest a more appropriate mode.

This is important because FTIAS is mode-driven. The selected mode determines which deterministic workflow and capability rules are applied.

The prompt guard does not silently replace engineering judgment. It helps the user notice when the selected mode and the written request may not match.

Confidence, coverage, and applicability controls

FTIAS includes structured analysis controls to help communicate the strength of interpretation for a result.

These controls include:

- deterministic confidence
- retrieval coverage
- applicability status
- warning level
- result strength
- downgrade or blocking reason

These controls are visible in reports and are intended to prevent weak or limited outputs from appearing stronger than they really are.

For example:

- A strong deterministic result may still be only partially applicable if certification corrections are missing
 - A general summary may have strong retrieval coverage but no deterministic confidence
 - A flutter-support pre-screen may raise high warnings while remaining non-clearance evidence
-

Report and PDF generation

FTIAS can export engineering reports from saved analysis jobs.

The report is designed to include:

- flight-test metadata
- dataset provenance
- analysis summary
- parameter statistics
- deterministic result
- standards cross-check
- assumptions and limitations
- applicability boundaries
- retrieved sources
- provenance statement

The important design rule is that reports should come from immutable saved artifacts, not from transient page text.

This supports traceability and reproducibility.

FRAT/mission risk workflow

FTIAS includes a FRAT/mission risk workflow.

The purpose of FRAT is to support structured mission-risk assessment. It is separate from normal AI analysis because it is a decision-support workflow with lifecycle states.

The FRAT workflow includes:

- draft creation
- risk input capture
- deterministic scoring
- hard-stop logic
- review/approval/rejection
- finalization
- export

A finalized FRAT should be treated as an auditable risk artifact.

The user should not interpret FRAT as a replacement for organizational approval authority. It is a structured support tool for documenting and reviewing risk factors.

Engineering boundaries

FTIAS must be understood as an engineering support system, not an automatic certification authority.

The system can help organize data, calculate bounded metrics, retrieve references, highlight anomalies, and generate traceable reports. However, final safety, performance, airworthiness, and certification conclusions must remain under qualified engineering review.

Deterministic does not mean unlimited

A deterministic calculation is stronger than a generic AI answer because it follows defined logic and uses measured data.

However, deterministic outputs are only valid inside their assumptions.

For example:

- takeoff ground-roll estimation depends on WOW and ground-speed signal quality
- landing rollout estimation depends on touchdown detection and speed trace integrity
- handling summaries depends on available synchronized control and response channels
- vibration and flutter-support results depend on sensor quality, sample cadence, and signal interpretation

If the required input data is missing or weak, the result must be downgraded, blocked, or marked as partial.

RAG support is advisory

Retrieval-augmented generation helps the system use uploaded references, standards, manuals, and technical documents.

However, RAG output remains context support. It can help explain, cross-check, or cite relevant material, but it should not invent unsupported engineering conclusions.

A source-rich answer may still be advisory if no deterministic computation supports the requested conclusion.

Some modes are screening only.

Some FTIAS modes are intentionally bound as screening or support workflows.

This is especially important for:

- vibration and loads
- flutter-support pre-screening
- general standards summaries
- early handling/control-response summaries

Screening results can identify potential areas of concern and recommend further review. They should not be treated as formal substantiation for certification.

For example, flutter-support pre-screening can identify suspicious oscillatory windows or high-speed concern concentrations, but it must not be interpreted as flutter clearance.

Certification corrections must be explicit.

Many flight-test results require correction factors before they can be used for certification-style conclusions.

Examples include:

- wind correction
- runway slope correction
- non-standard atmosphere correction
- aircraft weight/configuration correction
- screen-height correction
- pitot-static / air-data corrections

If those corrections are not implemented and applied, the report must say so.

The absence of these corrections does not render the result useless, but it limits how it may be used.

User responsibility

FTIAS supports engineering review, but it does not replace the engineer.

The user is responsible for:

- selecting the correct flight test
- selecting the correct dataset version
- choosing the correct analysis mode
- checking prompt/mode warnings
- verifying signal quality

- interpreting applicability boundaries
- reviewing retrieved sources
- validating conclusions before operational or certification use

The system is designed to make these responsibilities more visible, not to remove them.

System overview summary

FTIAS combines flight-test telemetry, document retrieval, deterministic analysis, workflow controls, and reporting into a single engineering-support environment.

The system's value comes from the connection between:

- uploaded data
- dataset versions
- selected analysis modes
- deterministic calculators
- document references
- saved analysis jobs
- reports
- FRAT risk artifacts

The central principle is traceability.

Every important output should answer:

- What flight test was used?
- What dataset version was used?
- What analysis mode was selected?
- What deterministic logic was applied?
- What sources were retrieved?
- What limitations apply?
- What artifact was saved or exported?

When used correctly, FTIAS helps the engineer move from raw test data to structured, reviewable, and reproducible engineering evidence.

CHAPTER 3 - Architecture and Component Connections

FTIAS is built as a layered web application. Each layer has a specific responsibility, and the system works correctly when each layer does only the work it is designed to do.

The main architectural principle is simple:

The frontend presents and collects user actions. The backend controls logic and data integrity. The database stores persistent truth. Docker runs and connects the services. External AI/document services are used only through backend-controlled workflows.

This separation is important because FTIAS handles engineering data, saved analyses, dataset provenance, reports, and mission-risk artifacts. These cannot safely depend only on browser state or temporary screen content.

Frontend

The frontend is the user-facing part of FTIAS. It runs in the browser and provides the visual interface used by the engineer.

The frontend is responsible for:

- displaying pages and navigation
- collecting user inputs
- showing flight tests and dataset versions
- displaying upload status and history
- rendering parameter lists and charts
- showing AI analysis results
- displaying warnings, controls, and provenance
- triggering report exports
- supporting the FRAT workflow interface

The frontend does not own the application's authoritative truth. It should not decide final data scope, ownership, deletion integrity, analysis provenance, or report immutability. Those responsibilities belong to the backend.

Examples of frontend pages include:

- Dashboard / Flight Test Detail
- Upload Data
- Parameters

- Document Library
- AI Standards Query
- FRAT / Mission Risk Workflow

The frontend communicates with the backend through HTTP API calls. It receives structured responses and renders them for the user.

For example, when the user selects a dataset version, the frontend sends that selection to the backend. The backend then returns the correct dataset-scoped parameters or analysis results.

Backend

The backend is the operational control layer of FTIAS. It receives API requests from the frontend, validates them, applies business rules, accesses the database, calls deterministic analysis modules, retrieves document context, and prepares reports.

The backend is responsible for:

- authentication and user ownership checks
- request validation
- flight-test management
- CSV ingestion
- dataset version creation and activation
- parameter and data retrieval
- deterministic engineering calculations
- AI analysis orchestration
- document retrieval and RAG support
- saved analysis job creation
- immutable report generation
- FRAT scoring and lifecycle control
- deletion integrity and foreign-key-safe cleanup

The backend is the correct place for application truth because it can enforce rules consistently.

For example, if a user requests an analysis for a flight test, the backend must confirm:

1. The user owns the flight test
2. The dataset version exists
3. The requested analysis mode is valid

4. The required data is available
5. The output should be saved with correct provenance

The frontend may display these choices, but the backend must enforce them.

Database

The database is the persistent memory of FTIAS. It stores the records that must survive page reloads, container restarts, and later user sessions.

FTIAS uses PostgreSQL as the primary relational database.

The database stores entities such as:

- users
- flight tests
- dataset versions
- ingestion sessions
- test parameters
- data points
- analysis jobs
- uploaded documents
- document chunks
- FRAT assessments

The database is especially important because many FTIAS workflows depend on traceability.

For example:

- A dataset version must remain linked to the flight test that created it
- An ingestion session must preserve the upload status and failure detail
- An analysis job must preserve the dataset version and output snapshot used at generation time
- A report must export from saved analysis artifacts, not from mutable screen content
- A finalized FRAT must preserve its scoring and approval state

Foreign keys protect these relationships. They also explain why deletion order matters. If another still references one record, the database may reject deletion until dependent records are removed or updated in the correct order.

Docker

Docker provides the runtime environment for FTIAS. It packages and runs the frontend, backend, and database services in a repeatable way.

Docker is not part of the engineering logic. It does not decide analysis results, dataset versions, or user permissions. Its role is to reliably start and connect services.

In a typical FTIAS development environment, Docker Compose manages:

- frontend container
- backend container
- PostgreSQL database container

The benefit of Docker is consistency. The system can be started with the same service definitions rather than manually configuring each component every time.

Docker also creates the service network. This allows the backend to reach PostgreSQL via the configured Compose service name, and the browser to reach the frontend and backend via the exposed ports.

Operationally, Docker matters because many common issues involve the relationship between:

- updated code
- container rebuilds
- container restarts
- database migrations
- service logs

A common example is a backend error after a new migration is added. The code may expect a new database column, but the database does not yet have it. The frontend may only show “Failed to fetch,” while the backend logs reveal the real schema mismatch.

External AI/Document Services

FTIAS uses AI and document-processing capabilities as support services. These are accessed through backend-controlled workflows.

The frontend should not directly manage model calls or document retrieval logic. Instead, the backend prepares the request, retrieves relevant context, applies capability rules, calls the model when needed, and saves the result as a structured artifact when appropriate.

External AI/document support is used for:

- AI Standards Query
- RAG-assisted analysis

- standards cross-checks
- narrative explanation
- recommendations and limitations
- source-backed context

However, FTIAS is designed so that AI is not the sole source of engineering truth.

Where deterministic calculators exist, they are treated as the primary source for the engineering metric. AI and retrieved sources support interpretation, explanation, and cross-checking.

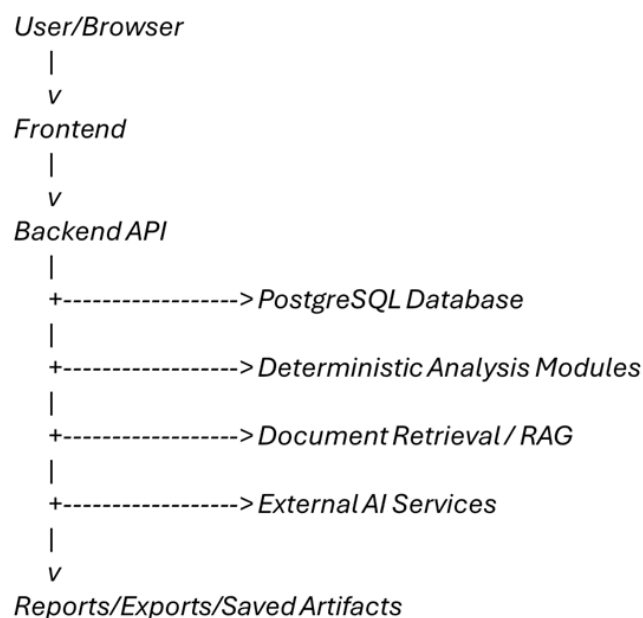
For example:

- takeoff ground roll is computed by deterministic logic
- landing rollout is computed by deterministic logic
- handling/control-response summaries are computed by bounded deterministic logic
- vibration and flutter-support screening produce deterministic indicators
- retrieved sources provide supporting context and cautionary interpretation

This separation helps prevent AI text from being mistaken for a validated engineering calculation.

Correct Connection Pattern

The intended connection pattern is:



The frontend does not bypass the backend.

The database does not directly serve the browser.

The AI layer does not directly own application state.

The backend coordinates all critical workflows.

This pattern keeps the system maintainable and safer for engineering use.

Main Runtime Services

A simplified runtime view is:

```
docker compose
|
+-- frontend service
|   - serves the web application
|   - renders pages and charts
|   - calls backend APIs
|
+-- backend service
|   - exposes REST APIs
|   - validates requests
|   - controls application logic
|   - performs deterministic analysis
|   - manages reports and FRAT
|
+-- postgres service
    - stores persistent records
    - protects relationships
    - preserves provenance
```

Depending on the environment, external AI services may run outside Docker or be accessed through configured API endpoints.

Component Responsibilities

The following table summarizes each component's main responsibility.

Component	Primary responsibility	Should not be responsible for
Frontend	User interaction, page rendering, charts, workflow display	Database ownership, final business rules, immutable provenance
Backend	Validation, logic, analysis, data control, reports, FRAT	Visual chart rendering, browser-only workflow memory
Database	Persistent records and relationships	Business interpretation or UI behavior
Docker	Runtime packaging and service networking	Engineering logic or data interpretation
AI / RAG	Context, explanation, standards support	Replacing deterministic calculations or final engineering judgment

This separation prevents confusion when troubleshooting. If a chart looks wrong, the issue may be frontend rendering, backend data retrieval, or dataset selection. If a report is missing provenance, the issue is likely in backend artifact generation. If an upload disappears from view but remains in the database, the issue may be due to API retrieval, schema mismatch, or frontend state handling.

Architecture by Workflow

The architecture can also be understood by following common workflows.

Upload workflow

1. The user selects a flight test and uploads a CSV in the frontend.
2. The frontend sends the file to the backend.
3. The backend validates the flight test and creates an ingestion session.
4. The backend creates a new dataset version.
5. The backend parses the CSV and inserts parameter/data records.
6. The database stores the new dataset version and data points.
7. The backend updates ingestion status.
8. The frontend displays success, failure, and dataset version information.

Parameter workflow

1. The user opens the Parameters page.
2. The frontend requests parameters for a selected flight test and dataset version.
3. The backend resolves the dataset scope.

4. The backend returns available parameters and data series.
5. The frontend renders charts and overlays.

AI analysis workflow

1. The user selects an analysis mode and writes a prompt.
2. The frontend sends the selected mode, prompt, flight test, and dataset version to the backend.
3. The backend applies prompt-to-mode guard logic.
4. The backend resolves the capability and required data.
5. The backend runs deterministic analysis where supported.
6. The backend retrieves document context where useful.
7. The backend generates the analysis response.
8. The backend saves an immutable analysis job.
9. The frontend displays the result and analysis job ID.

Report workflow

1. The user exports a saved analysis.
2. The frontend requests report export by analysis job ID.
3. The backend loads the saved analysis artifact.
4. The backend builds the report from saved snapshots and provenance.
5. The frontend receives the downloadable PDF.

FRAT workflow

1. The user creates or opens a FRAT assessment.
2. The frontend displays the FRAT form and lifecycle controls.
3. The backend stores the draft assessment.
4. The backend computes deterministic risk scoring.
5. The backend applies hard-stop rules.
6. The user approves, rejects, or finalizes according to allowed transitions.
7. The backend preserves the final snapshot.
8. The user exports the FRAT artifact.

Why This Architecture Matters

FTIAS handles engineering data that must remain interpretable after the immediate browser session ends.

That requires:

- backend-enforced ownership
- immutable dataset versions
- persisted ingestion sessions
- saved analysis jobs
- stored provenance
- deterministic calculation traces
- report export from saved artifacts
- FRAT lifecycle states
- clear applicability boundaries

Without these controls, the system would be only a temporary analysis screen. With these controls, it becomes a traceable engineering-support environment.

The architecture, therefore, supports three major goals:

1. **Usability**
The engineer can upload, inspect, analyze, and export through a clear interface.
2. **Traceability**
The system can explain which dataset, analysis mode, sources, and artifacts produced an output.
3. **Maintainability**
Each layer has a clear role, making future changes safer and easier to test.

Chapter summary

FTIAS is built from connected but distinct components.

The frontend provides the user interface.

The backend enforces rules and coordinates workflows.

The database stores persistent truth.

Docker runs the services in a repeatable environment.

External AI and document services support interpretation and reference retrieval.

The most important architectural rule is that critical truth belongs in the backend and database, not in the temporary browser state.

This rule supports the system's broader purpose: producing traceable, reproducible, and reviewable engineering outputs from flight-test data.

CHAPTER 4 - Installation and Docker Environment

This chapter explains how to run FTIAS in a local development or test environment using Docker Compose. It focuses on practical operation: preparing the environment, starting services, rebuilding containers, applying migrations, checking logs, and understanding common startup problems.

FTIAS depends on several moving parts working together. The frontend, backend, database, migrations, environment variables, and external AI configuration must be aligned. Most runtime issues are easier to solve when these parts are understood as one environment rather than separate pieces.

Prerequisites

Before running FTIAS, the development environment should have the required tools installed and available from the command line.

Typical prerequisites include:

- Git
- Docker Desktop or Docker Engine
- Docker Compose
- Node.js / package manager support, if running frontend tasks outside Docker
- Python support, if running backend tasks outside Docker
- access to the project repository
- required environment variables
- access credentials for external AI or document services, if used

In the normal Docker workflow, Docker Compose is responsible for starting the main application services. However, local tools may still be useful for development, testing, dependency inspection, or debugging.

The exact dependency versions should follow the project repository files, such as:

- Dockerfiles
- docker-compose configuration
- backend dependency files
- frontend dependency files
- environment templates or .env files

If the application fails to start, first confirm that Docker is running and that the repository is on the expected branch.

Repo Structure

The repository is organized into frontend, backend, migration, and documentation areas.

A simplified view is:

```
flight-test-interactive-analysis-suite/
|
+-- backend/
|   +-- app/
|   +-- routers/
|   +-- tests/
|   +-- migrations/
|
+-- frontend/
|   +-- src/
|   +-- components/
|   +-- pages/
|   +-- services/
|
+-- docs/summary files
|
+-- docker-compose.yml
+-- TODO.md
```

The backend contains the API, data models, routing logic, deterministic calculators, document retrieval logic, report generation, and FRAT workflow.

The frontend contains the user interface, pages, chart components, API client, and workflow screens.

The migrations define database schema changes that must be applied to PostgreSQL before backend code that expects those schema changes can run correctly.

The planning and summary files document the roadmap, implementation history, and accepted design decisions.

Docker Services

A typical FTIAS Docker Compose environment includes at least these services:

- frontend
- backend
- postgres

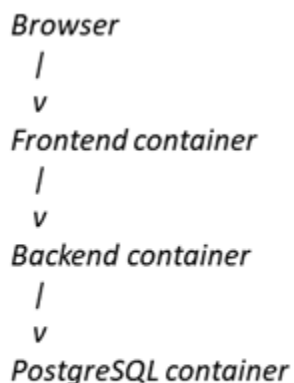
The frontend service serves the browser application.

The backend service exposes the API and performs application logic.

The PostgreSQL service stores persistent data.

Depending on the exact project configuration, there may also be additional services for document processing, storage, queues, or future background jobs. The current manual focuses on the core services.

A simplified service relationship is:



The backend may also call external AI services or APIs, depending on the environment variables and configured credentials.

Starting the Application

From the repository root, the normal command to start the application is:

- “docker compose up -d”

This starts the configured services in detached mode.

After starting, check the containers:

- “docker compose ps”

This should show the frontend, backend, and database containers running.

If a service exists or restarts repeatedly, check its logs.

For the backend:

- “docker compose logs -f backend”

For the frontend:

- “docker compose logs -f frontend”

For PostgreSQL:

- “docker compose logs -f postgres”

Stopping the Application

To stop the running services:

- “docker compose down”

This stops and removes the running containers created by Docker Compose.

Depending on Docker volume configuration, database data may be preserved in Docker volumes. Do not remove volumes unless a data reset is intended.

A more aggressive cleanup, such as deleting volumes, should be done carefully because it may remove the PostgreSQL data.

Restart vs Rebuild

A frequent operational question is whether to restart or rebuild.

Restart

A restart is usually enough when:

- code is already mounted into the container
- only the runtime state needs refreshing
- backend or frontend service needs to reload
- no dependencies changed
- no Dockerfile changed

Command:

- “docker compose restart backend frontend”

or for one service:

- “docker compose restart backend”

Rebuild

A rebuild is recommended when:

- dependencies changed
- Dockerfile changed
- frontend build behavior changed
- backend image contents changed
- The application does not reflect new code after a restart
- Codex or development work changed both backend and frontend build paths

Command:

- “docker compose up -d --build backend frontend”

For a full rebuild:

- `"docker compose up -d --build"`

In practice, after significant feature work touching backend and frontend, this is usually safest:

- `"docker compose up -d --build backend frontend"`

Migrations

Database migrations are required when backend code expects new database tables or columns to be created.

A migration mismatch is one of the most common causes of errors after new development work.

Typical symptoms include:

- frontend shows "Failed to fetch"
- backend returns 500 errors
- backend logs show missing column errors
- backend logs show missing table errors
- document library or analysis pages fail after a code update

The reason is simple: the backend code has changed, but the PostgreSQL schema has not yet been updated.

Applying a migration

A common PowerShell pattern for applying a migration is:

- `Get-Content backend/migrations/<migration_file>.sql -Raw | docker compose exec -T postgres sh -lc 'psql -v ON_ERROR_STOP=1 -U "$POSTGRES_USER" -d "$POSTGRES_DB"'`

Replace `<migration_file>.sql` with the correct migration filename.

Example:

- `Get-Content backend/migrations/20260424_add_prompt_mode_guard_snapshot.sql -Raw | docker compose exec -T postgres sh -lc 'psql -v ON_ERROR_STOP=1 -U "$POSTGRES_USER" -d "$POSTGRES_DB"'`

After applying a migration, restart the backend:

- `docker compose restart backend`

If frontend contracts also changed, rebuild both backend and frontend:

- `docker compose up -d --build backend frontend`

Migration rule

The rule is:

Apply the migration before running backend code that depends on it.

If this order is reversed, the backend may fail because it asks PostgreSQL for columns or tables that do not yet exist.

Logs

Logs are the first place to look when the application behaves unexpectedly.

Backend logs

Backend logs are usually the most important when the frontend shows a generic error.

Command:

- `docker compose logs -f backend`

Backend logs can reveal:

- missing database columns
- missing tables
- foreign-key violations
- failed document retrieval
- AI service errors
- failed report export
- validation errors
- startup failures

Frontend logs

Frontend logs are useful when the browser interface fails to build or serve.

Command:

- `docker compose logs -f frontend`

Frontend logs can reveal:

- build errors
- missing modules
- route errors
- frontend service startup problems

PostgreSQL logs

PostgreSQL logs are useful for database startup, connection, and schema problems.

Command:

- `docker compose logs -f postgres`

PostgreSQL logs can reveal:

- authentication issues
- database startup problems
- schema or query errors
- connection failures

Environment Variables

FTIAS may require environment variables for database connections, authentication, access to AI services, and other runtime settings.

These are usually provided through a .env file or Docker Compose environment configuration.

Typical categories include:

- database user
- database password
- database name
- backend API settings
- frontend API base URL
- AI provider key or endpoint
- document-processing configuration

Environment variables must be consistent across services. For example, the backend must know how to reach PostgreSQL using the service name and credentials configured in Docker Compose.

If environment variables are missing or incorrect, the backend may fail at startup or fail when calling external services.

Common Runtime Problems

“Failed to fetch”

This usually means the frontend could not complete an API call.

Possible causes include:

- backend is not running
- backend crashed during startup
- backend returned a 500 error
- migration was not applied
- frontend API base URL is incorrect

- network or port configuration is wrong

Recommended check:

- `docker compose logs -f backend`

Then test the backend API documentation page if available, such as:

`http://localhost:8000/docs`

Missing database column

This usually means a migration was not applied.

Example type of error:

`column documents.authority_type does not exist`

Fix:

1. apply the relevant migration
2. restart backend
3. Test the affected page again

Foreign-key violation

This means the database is protecting relationships between records.

Possible causes include:

- deleting records in the wrong order
- leaving dependent records attached
- A new entity was added, but the deletion logic was not updated

Fixing this usually requires changes to backend logic, not frontend code.

Frontend does not show new UI changes

Possible causes:

- frontend container was not rebuilt
- browser cache
- old frontend bundle still running
- code changed, but service was only restarted when a rebuild was required

Recommended command:

- `docker compose up -d --build frontend`

If backend contracts also changed:

- `docker compose up -d --build backend frontend`

The backend does not reflect the new logic

Possible causes:

- The backend was not rebuilt or restarted
- The wrong branch is running
- The container is using an older image
- migration mismatch prevents startup

Recommended command:

- `docker compose up -d --build backend`

Then check logs:

- `docker compose logs -f backend`

Development Validation Commands

After backend work, run relevant backend tests.

Examples:

- `pytest backend/tests/test_documents_tenancy.py -q`
- `pytest backend/tests/test_analysis_mode_routing.py -q`
- `pytest backend/tests/test_deterministic_calculators.py -q`
- `pytest backend/tests/test_frat_workflow.py -q`

After frontend work, run the frontend build.

Example:

- `pnpm -C frontend run build`

A common validation pattern after full-stack changes is:

- `pytest backend/tests/test_analysis_mode_routing.py backend/tests/test_documents_tenancy.py -q`
- `pnpm -C frontend run build`

The exact tests depend on which area changed.

Recommended Operational Sequence After New Code

After pulling or receiving new code, use this general sequence:

1. Confirm the branch and commit.
2. Check whether new migrations were added.
3. Apply required migrations.
4. Rebuild affected services.
5. Check service status.

6. Check backend logs.
7. Open the app.
8. Test the feature manually.

Example:

- `git status`
- `git log --oneline -5`
- `docker compose ps`

If a migration exists:

- `Get-Content backend/migrations/<migration_file>.sql -Raw | docker compose exec -T postgres sh -lc 'psql -v ON_ERROR_STOP=1 -U "$POSTGRES_USER" -d "$POSTGRES_DB"'`

Then rebuild:

- `docker compose up -d --build backend frontend`

Check logs:

- `docker compose logs -f backend`

Safe Handling of Data

The PostgreSQL database contains the working project data.

Be careful with commands that remove volumes or reset the database.

Commands that only restart or rebuild services usually preserve database data.

Commands that remove Docker volumes may delete database content.

Before deleting volumes, confirm that data loss is intended or that a backup exists.

Chapter Summary

The Docker environment is the runtime shell that allows FTIAS services to run together.

The frontend, backend, and PostgreSQL services must be aligned with the current code, dependencies, environment variables, and database schema.

The most important operational rules are:

- rebuild when image-level or dependency changes occur
- restart when only service refresh is needed
- apply migrations before running backend code that expects the new schema
- check backend logs when the frontend shows “Failed to fetch”
- Avoid deleting Docker volumes unless the database reset is intentional

Understanding these rules makes FTIAS easier to operate, troubleshoot, and maintain.

CHAPTER 5 - Frontend Operation

The frontend is the part of FTIAS that the user interacts with directly in the browser. It provides the pages, forms, charts, workflow panels, warnings, and export actions used during normal operation.

The frontend does not own the application's final truth. It displays and organizes information, but the backend remains responsible for validation, ownership, dataset scope, analysis routing, saved artifacts, and report generation.

The main purpose of the frontend is to help the user move through the engineering workflow clearly:

1. create or select a flight test
2. upload or select a dataset version
3. inspect parameters and charts
4. run AI or deterministic analysis
5. review warnings, limitations, and provenance
6. export reports
7. create or review FRAT / mission-risk assessments

The frontend should always make the current context visible: the selected flight test, selected dataset version, selected analysis mode, and any warnings or limitations returned by the backend.

Pages

FTIAS is organized around several main frontend pages. Each page supports a different part of the workflow.

Dashboard/Flight Test Detail

The Dashboard or Flight Test Detail page is the main working area for a selected flight test.

Typical functions include:

- viewing flight-test metadata
- reviewing available dataset versions
- selecting or activating a dataset version
- viewing parameter summaries
- running AI analysis
- reopening saved analysis jobs by ID
- exporting reports
- seeing dataset provenance and analysis-mode information

This page is usually where the user moves from raw uploaded data into engineering interpretation.

The most important point is that analysis actions on this page must be tied to the selected dataset version. If a saved analysis is reopened, the UI should show the dataset provenance stored with that saved job, even if the currently selected dataset is different.

Upload Data

The Upload Data page is used to upload CSV files into a selected flight test.

Typical functions include:

- selecting a flight test
- uploading CSV data
- viewing ingestion status
- viewing upload history
- viewing dataset versions
- identifying the active dataset version

After a successful upload, the backend creates a new immutable dataset version. The frontend displays that version to the user.

Upload History and Dataset Versions are related but not identical:

- Upload History shows the operational history of uploaded files and ingestion sessions.
- Dataset Versions show the usable immutable datasets created from successful uploads.

The frontend should display the real dataset version label, such as v1, v2, or v3, not the raw database ID.

Parameters

The Parameters page supports detailed signal exploration.

Typical functions include:

- searching available parameters
- selecting parameters for plotting
- marking parameters as favourites
- saving parameter sets
- applying saved parameter sets
- plotting selected parameters
- using threshold overlays

- using event markers
- comparing dataset versions
- exporting chart images

This page is especially important for large datasets with many channels.

The user should use this page to inspect signal quality, detect trends, compare runs, and prepare better analysis prompts.

For example, before asking for a takeoff or handling-quality analysis, the user can inspect whether the expected channels are present and reasonable.

Document Library

The Document Library page manages uploaded reference documents.

Typical functions include:

- viewing uploaded documents
- checking document availability
- supporting retrieval-augmented analysis
- providing source material for AI Standards Query and AI Analysis

Documents are used by the retrieval system. The backend indexes and retrieves relevant document chunks when answering standards queries or supporting analysis modes.

If the Document Library shows a fetch error, the real cause is often backend-side. Common causes include backend service failure, missing migrations, or database schema mismatch.

AI Standards Query

The AI Standards Query page is used for document-centered questions.

Typical use cases include:

- asking what a standard says about a topic
- finding relevant sections in uploaded manuals
- comparing guidance across documents
- supporting interpretation of engineering requirements

This workflow is different from flight-test AI Analysis.

AI Standards Query focuses mainly on uploaded documents and retrieved references. It is useful for standards interpretation but should not be treated as a deterministic calculation workflow.

FRAT/Mission Risk Workflow

The FRAT page supports structured mission-risk assessment.

Typical functions include:

- creating a FRAT draft
- entering risk factors
- scoring the assessment
- reviewing hard-stops
- approving or rejecting
- finalizing the assessment
- exporting the finalized artifact

The FRAT workflow is stateful. This means the backend tracks whether an assessment is draft, scored, approved, rejected, or finalized.

The frontend displays the workflow, but the backend enforces the valid transitions.

Navigation

The frontend navigation should help the user move through the normal engineering workflow.

A typical workflow is:

1. open or create a flight test
2. upload CSV data
3. confirm dataset version creation
4. inspect parameters
5. run analysis
6. export report
7. create or review FRAT if needed

The exact order may vary, but the user should normally avoid running analysis before confirming that the correct dataset version is active or selected.

Navigation should also preserve context where practical. For example, if the user is working inside a specific flight test, moving between Upload, Parameters, and Flight Test Detail should not create confusion about which flight test or dataset version is being used.

Dataset Selection

Dataset selection is one of the most important frontend operations.

A flight test may have more than one dataset version. For example:

- v1 may be the first upload
- v2 may be a corrected upload
- v3 may be a larger or more complete dataset

The selected dataset version determines which data is used for:

- parameter lists
- charts
- AI analysis
- deterministic calculations
- report generation

The active dataset is the default dataset version for the flight test. However, the user may select historical versions for review or comparison.

The frontend should clearly show:

- available dataset versions
- selected dataset version
- active dataset version
- dataset status
- row count or data-point count where available

Saved analysis jobs preserve their own dataset provenance. If a user reopens an old analysis job, the job may refer to a dataset version that is not currently selected. In that case, the UI should clearly show the saved job's dataset version.

AI Mode Selection

AI Analysis is mode-driven. The selected analysis mode tells the backend which capability and deterministic workflow should be used.

Current frontend analysis modes include:

- Takeoff Performance
- Landing Performance
- Performance / Climb / Air Data

- Handling / Control Response
- Vibration & Loads
- Flutter Support Pre-screen
- General Summary

Each mode has a different purpose.

Takeoff Performance

Used for takeoff ground-roll and liftoff-related analysis.

Typical required or useful parameters include:

- WOW / Weight on Wheels
- ground speed
- acceleration
- timestamp

The output is a bounded deterministic estimate unless certification corrections are explicitly implemented and applied.

Landing Performance

Used for landing rollout and deceleration-related analysis.

Typical required or useful parameters include:

- WOW / touchdown cues
- ground speed
- deceleration
- timestamp

The output is a bounded landing rollout estimate or a partial result if detection is weak.

Performance/Climb/ Air Data

Used for general performance trends and air-data support.

Typical parameters include:

- altitude
- vertical speed
- ground speed

- CAS
- TAS
- Mach
- pressure altitude
- temperature

This mode can include atmosphere and air-data support such as density altitude, ISA-related indicators, and speed consistency checks.

It is not a full certification performance-reduction package unless the necessary correction models are explicitly implemented.

Handling/Control Response

Used for control-input and aircraft-response analysis.

Typical parameters include:

- aileron deflection
- elevator deflection
- rudder deflection
- stick position
- roll rate
- pitch rate
- yaw rate
- roll angle
- pitch angle

This mode supports bounded handling/control-response observations. It is not a formal handling-qualities certification determination.

Vibration & Loads

Used for bounded vibration and load-related screening.

Typical parameters include:

- accelerations
- angular rates

- airspeed-rate channels
- response-sensitive telemetry

This mode can summarize grouped channels, dominant contributors, regime segmentation, anomaly/event windows, and bounded frequency-domain cues where supported.

It is screening only. It is not loads substantiation or flutter clearance.

Flutter Support Pre-screen

Used for bounded flutter-support screening.

Typical parameters include:

- angular rates
- accelerations
- airspeed / Mach / altitude context
- vibration-sensitive channels
- control or response channels where relevant

This mode may flag concern windows, high-speed regime concentrations, or repeated frequency cues.

It must not be interpreted as flutter clearance, modal-identification certification, or aeroelastic substantiation.

General Summary

Used for broad summaries and advisory context.

This mode is useful when the user wants:

- executive overview
- general observations
- recommendations
- standards context
- broad review

General Summary is not a substitute for a specific deterministic mode when the question clearly asks for takeoff, landing, performance, handling, vibration, or flutter-related metrics.

Prompt-to-Mode Warnings

The frontend displays prompt-to-mode warnings when the user's written prompt appears to conflict with the selected analysis mode.

Example:

If the selected mode is Takeoff Performance, but the prompt says:

"Analyze the available aileron deflection and stick position data."

The system may detect that the prompt is more related to Handling/Control Response than takeoff.

The frontend should then show:

- detected intent
- selected mode
- mismatch severity
- matched prompt cues
- suggested mode

This warning helps prevent the user from accidentally running an analysis under the wrong capability.

The warning does not remove user responsibility. It helps the user verify whether the selected mode is correct.

Saved Analysis Reopen

FTIAS supports reopening saved analysis jobs by ID.

A saved analysis job is an immutable artifact. It stores the analysis result, prompt, selected mode, dataset version, retrieved sources, parameter statistics snapshot, and controls.

When a saved analysis is reopened, the frontend should display:

- analysis job ID
- original prompt
- selected mode
- executed mode, if different
- dataset version used by the job
- retrieved sources snapshot
- confidence/coverage / applicability controls
- prompt-to-mode guard information

- saved narrative text

This matters because a reopened job should reflect what was originally analyzed, not what happens to be selected now.

For example, if the current active dataset is v3 but the saved analysis was created against v1, the UI should make that difference visible.

Report Export from the Frontend

The frontend can request a report export for saved analysis jobs.

The correct export flow is:

1. The user runs or reopens an analysis
2. backend confirms the saved analysis job exists
3. frontend requests export by analysis job ID
4. The backend generates a PDF from the saved artifact
5. The browser downloads the PDF

The frontend should not generate the official engineering report from temporary screen text.

This preserves report reproducibility.

FRAT Page

The FRAT page supports the mission-risk workflow.

A typical FRAT user sequence is:

1. Create a draft
2. Enter mission or test risk inputs
3. link supporting analysis if applicable
4. Score the assessment
5. Review total score and category scores
6. Review hard stops
7. approve or reject
8. finalize
9. export

The frontend should clearly show the FRAT lifecycle state.

Possible states may include:

- draft
- scored
- needs review
- approved
- rejected
- finalized

The exact state labels depend on backend implementation, but the principle is that the user should know whether the FRAT is still editable or already finalized.

A finalized FRAT should be treated as an immutable artifact.

User Checklist for Frontend Operation

Before running an analysis, the user should check:

1. Is the correct flight test selected?
2. Is the correct dataset version selected?
3. Are the required parameters present?
4. Is the selected analysis mode appropriate?
5. Did the prompt-to-mode guard show any warning?
6. Are the retrieved sources relevant if the analysis uses RAG?
7. Are confidence, coverage, and applicability controls acceptable for the intended use?
8. Is the result being used only within its stated limitations?

Before exporting a report, the user should check:

1. Is the correct analysis job open?
2. Does the analysis job reference the expected dataset version?
3. Are the limitations and applicability boundaries clear?
4. Are the retrieved sources and narrative citations reasonable?
5. Is the report intended for engineering review, not automatic approval?

Before finalizing a FRAT, the user should check:

1. Are all mission-risk inputs complete?
2. Are hard stops resolved or accepted according to procedure?
3. Are supporting analyses linked or referenced where useful?

4. Has the reviewer checked the result strength and warning levels?
5. Is the final state intentional?

Common Frontend Issues

Page shows “Failed to fetch”

This usually means the frontend could not complete an API call.

The real cause is usually in the backend logs. Common causes include:

- backend not running
- backend startup failure
- missing database migration
- backend 500 error
- incorrect API URL
- network or service routing problem

Recommended action:

- check backend logs
- confirm backend is running
- confirm migrations are applied
- refresh the page after backend recovery

New UI changes are not visible.

Possible causes include:

- frontend container was not rebuilt
- browser cache still has an older bundle
- the wrong branch or commit is running
- frontend build failed

Recommended action:

- docker compose up -d --build frontend

If backend contracts changed too:

- docker compose up -d --build backend frontend

The dataset label looks wrong.

Dataset labels should show the real version label, such as v1, v2, or v3.

If the UI shows a label that appears to be a raw database ID, it may indicate a frontend display issue or an API contract issue.

The Dataset Versions panel and Upload History should display consistent user-facing dataset labels.

Saved parameter sets disappear.

Saved parameter sets are intended to persist across navigation for the same flight test.

If they appear missing, check:

- whether the same flight test is selected
 - whether the current dataset has the selected parameters available
 - whether browser storage was cleared
 - whether a frontend persistence issue has occurred
-

Chart export does not work.

Possible causes include:

- browser download blocked
- chart rendering issue
- frontend export path failure
- unsupported chart state
- browser cache issue

Recommended action:

- refresh the page
 - test with fewer parameters
 - check browser console if available
 - rebuild the frontend if the issue appeared after a code change
-

Frontend Operation Summary

The frontend is the user's working surface for FTIAS.

It supports:

- flight-test selection
- dataset selection
- upload monitoring
- parameter exploration
- chart review
- AI analysis
- saved analysis reopen
- report export
- FRAT workflow

The frontend should always make context visible. The user should be able to understand which flight test, dataset version, analysis mode, saved job, or FRAT state is active.

The backend remains the authoritative source of truth, but the frontend is responsible for making that truth understandable and usable.

CHAPTER 6 - Backend Operation

The backend is the operational control layer of FTIAS. It receives requests from the frontend, validates them, applies business rules, reads and writes the database, runs deterministic calculations, retrieves document context, creates saved artifacts, and prepares reports.

The frontend is the user interface, but the backend is where the main application truth is enforced.

The backend is responsible for:

- user authentication and ownership checks
- flight-test creation and deletion
- CSV ingestion and dataset versioning
- parameter and data retrieval
- document retrieval and RAG support
- analysis-mode routing
- deterministic engineering calculations
- confidence, coverage, and applicability controls
- prompt-to-mode guard behavior
- saved analysis job persistence
- PDF report generation
- FRAT scoring and workflow state transitions
- database integrity during deletion

The backend's most important role is to ensure that FTIAS outputs remain traceable, reproducible, and technically bounded.

Routers

Backend routers expose the API endpoints used by the frontend. A router groups related API operations into a logical area.

Typical router responsibilities include:

- receiving HTTP requests
- validating path and body parameters
- checking the current user
- calling database queries or service functions

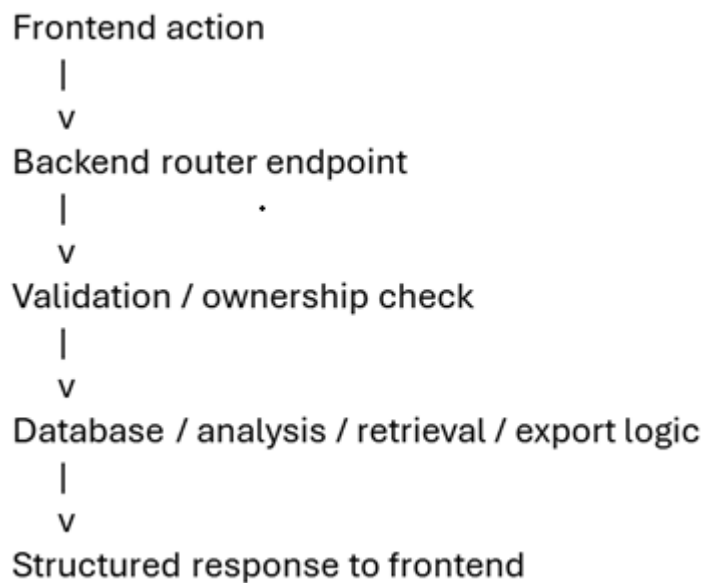
- returning structured JSON responses
- returning file exports when needed

The frontend should not bypass these routers. All user-facing workflows should pass through backend API endpoints.

Examples of backend API areas include:

- flight-test management
- document management
- AI Standards Query
- AI Analysis
- dataset versions
- parameters and data
- report export
- FRAT workflow

A simplified request path is:



The router is not only a communication endpoint. It is also a control point. It should prevent invalid, unauthorized, or unsafe requests from reaching deeper application logic.

Services

Services are backend logic areas that perform the work behind the API routes.

A service may perform operations such as:

- parsing uploaded CSV files
- creating ingestion sessions
- creating dataset versions
- inserting data points
- computing parameter statistics
- preparing document retrieval context
- running deterministic analysis
- building saved analysis artifacts
- generating PDF reports
- evaluating FRAT scores and hard-stops

Not every backend function needs to be called a “service,” but the idea is important: the backend should separate request handling from reusable business logic where possible.

A good backend design avoids placing all logic directly inside router functions. Routers should coordinate. Services and modules should perform specialized logic.

For example:

- a router receives an upload request
- ingestion logic parses the file and creates data records
- dataset-version logic creates immutable dataset versions
- database logic persists the result
- the router returns the final status to the frontend

This separation makes the system easier to test and maintain.

Deterministic Modules

Deterministic modules are backend components that compute engineering results using explicitly programmed logic.

These modules are important because FTIAS should not rely on free-form AI text for primary engineering metrics when deterministic computation is available.

Current deterministic or bounded deterministic areas include:

- takeoff ground-roll estimation
- landing rollout estimation
- performance trend analysis

- atmosphere / air-data support
- handling / control-response analysis
- buffet/vibration screening
- flutter-support pre-screening
- FRAT scoring

The deterministic modules should return structured results, not only narrative text.

A typical deterministic result should include:

- whether the calculation was available
- metrics produced
- channels used
- assumptions
- limitations
- applicability boundaries
- warning or concern indicators
- reason for partial or unavailable output

This allows the same backend result to support:

- AI Analysis display
- saved analysis jobs
- PDF reports
- future UI improvements
- FRAT or risk review logic

The key rule is:

Deterministic outputs are stronger than AI narratives within their defined boundaries, but they are not automatically certification-ready.

For example, a takeoff calculator may estimate ground roll to liftoff from WOW and ground-speed data. That result is useful, but it is not a certification-corrected takeoff distance unless wind, runway slope, atmospheric conditions, screen height, and other required corrections are applied.

Saved Artifact Model

FTIAS relies on saved artifacts to make analysis and reporting reproducible.

The most important saved artifact is the Analysis Job.

An analysis job stores the result of an AI Analysis request. It should preserve the analysis as it existed when it was generated.

A saved analysis job may include:

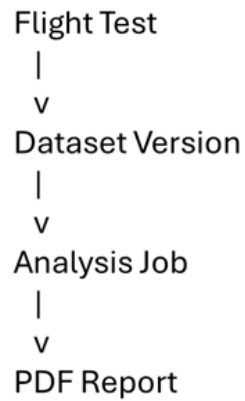
- flight test ID
- dataset version ID
- selected analysis mode
- executed analysis mode, if guarded or downgraded
- user prompt
- model metadata
- deterministic result
- analysis text
- retrieved sources snapshot
- parameter statistics snapshot
- prompt-to-mode guard snapshot
- confidence/coverage/applicability controls
- output hash
- created timestamp
- user ownership information

This matters because flight-test data can change over time. New dataset versions may be uploaded, active dataset selections may change, and documents may be added. A saved analysis should not drift because the current state changed later.

When the user reopens an analysis job, the backend should return the saved artifact rather than regenerate a new answer without traceability.

When the user exports a report, the backend should build the report from the saved analysis artifact.

This creates a reproducible chain:



The report should therefore be able to explain exactly which flight test and dataset version produced the output.

Deletion Integrity

Deletion is simple in the frontend but complex in the backend.

When a user deletes a flight test, the backend must safely remove related records in an order that respects database relationships.

A flight test may be linked to:

- data points
- dataset versions
- ingestion sessions
- analysis jobs
- FRAT assessments
- active dataset pointer
- related provenance records

The database uses foreign keys to protect these relationships. If the backend deletes records in the wrong order, PostgreSQL may reject the operation with a foreign-key violation.

The backend must therefore delete dependent records in a safe, sequenced manner.

A simplified safe deletion concept is:

1. Confirm the user owns the flight test
2. Clear active dataset pointers if needed
3. Delete dependent artifacts such as FRAT assessments and analysis jobs
4. delete data points

5. delete dataset versions
6. Delete ingestion sessions
7. Delete the flight test record

The exact implementation may change as the data model grows, but the principle remains the same:

Delete dependent records before deleting the parent record they reference.

This is why deletion logic belongs in the backend. The frontend can request deletion of a flight test, but it cannot safely manage all database dependencies on its own.

Routing and Controls

FTIAS uses routing logic to decide how an analysis request should be handled.

When the user runs AI Analysis, the backend considers:

- selected analysis mode
- user prompt
- available dataset parameters
- capability catalog
- prompt-to-mode guard
- deterministic calculator availability
- document retrieval support
- confidence/coverage/applicability controls

The backend must make sure that the analysis mode and the requested output are aligned.

For example:

- Takeoff prompts should use the takeoff capability
- Landing prompts should use the landing capability
- aileron/stick/control-response prompts should use handling qualities
- vibration prompts should use vibration screening
- Flutter prompts should use Flutter Support pre-screening
- Broad summary prompts may use the general mode

The prompt-to-mode guard helps detect when the selected mode and written prompt do not match.

For example, if the user selects **Takeoff Performance** but asks about aileron deflection and stick position, the backend can detect that the prompt intent is closer to handling qualities.

The backend may then return warning information such as:

- inferred prompt intent
- selected mode
- mismatch severity
- matched prompt cues
- suggested mode
- execution mode

This protects the user from accidentally treating an output as valid for the wrong engineering domain.

Capability Catalog

The capability catalog is the backend source of truth for what FTIAS can and cannot do.

It defines important information such as:

- capability key
- capability status
- authority classification
- required inputs
- blocked conditions
- downgrade conditions
- applicability boundaries
- whether deterministic logic exists
- whether RAG support is secondary or primary

The capability catalog prevents the UI or AI narrative from implying unsupported capability.

For example:

- Takeoff may be implemented as a deterministic estimate with limitations
- general summary may be advisory
- Flutter support may be implemented only as pre-screening
- vibration/load analysis may be screening only
- certification-level conclusions may remain unsupported unless the required models and data exist

The backend should use the capability catalog when routing analysis requests and when generating warnings or applicability statements.

Analysis Controls

Analysis controls describe how strongly the result should be interpreted.

These controls include:

- result strength
- deterministic confidence
- retrieval coverage
- applicability status
- warning level
- downgrade or blocking reason

These controls help prevent weak outputs from appearing stronger than they are.

For example:

- A deterministic takeoff result may have high deterministic confidence but still be only partially applicable because certification corrections were not applied
- A general summary may have strong retrieval coverage but an unavailable deterministic confidence
- A flutter-support pre-screen may produce high warning severity but remain non-clearance evidence
- A vibration result with no retrieved sources may remain useful as deterministic screening, but weak as a standards-supported conclusion

The backend computes these controls and includes them in analysis responses and saved artifacts.

The frontend and report generator then display them to the user.

Document Retrieval and RAG Support

The backend manages document retrieval for AI Standards Query and RAG-assisted analysis.

The retrieval process may use:

- document chunks
- source metadata
- authority type
- document revision
- domain tags
- capability tags

- analysis mode
- query text

Mode-aware retrieval helps select more relevant sources.

For example:

- Takeoff analysis should prefer takeoff/performance documents
- handling analysis should prefer handling-qualities or control-response references
- Flutter-support analysis should prefer flutter, aeroelastic, vibration, or flight-test risk references

However, retrieval is still supported by logic. It helps with standards context and interpretation, but it should not replace deterministic calculation where deterministic logic is implemented.

The backend should preserve retrieved-source snapshots in saved analysis jobs so reopened reports remain traceable.

Report Generation

The backend generates PDF reports from saved analysis artifacts.

The report should not depend on the temporary frontend state.

The backend report generation process typically uses:

- saved analysis job
- flight-test metadata
- dataset provenance
- parameter statistics snapshot
- deterministic result
- analysis controls
- prompt guard snapshot
- retrieved source snapshot
- narrative text
- output hash

This allows the report to include a provenance statement explaining where the result came from.

The important rule is:

A report should be exported from a saved artifact, not from the current mutable page state.

This supports reproducibility and auditability.

FRAT Backend Workflow

The FRAT workflow is backend-controlled because it involves risk scoring, hard stops, lifecycle states, and finalization.

The backend is responsible for:

- creating FRAT draft records
- storing user-entered risk factors
- computing deterministic risk scores
- evaluating hard-stops
- enforcing allowed state transitions
- approving or rejecting assessments
- finalizing assessments
- exporting immutable FRAT artifacts

A finalized FRAT should become an auditable record.

The frontend displays the workflow, but the backend must enforce whether an action is valid.

For example, the backend should prevent invalid transitions such as editing a finalized FRAT as if it were still a draft.

Backend Operation Summary

The backend is the control center of FTIAS.

It enforces:

- ownership
- validation
- dataset scope
- deterministic analysis logic
- document retrieval
- analysis controls
- prompt-to-mode alignment
- saved artifact immutability
- report provenance

- FRAT state transitions
- deletion integrity

The frontend helps the user interact with the system, but the backend ensures the system remains consistent, traceable, and safe for engineering support.

A healthy FTIAS backend should answer these questions for every important output:

- Who owns the flight test?
- Which dataset version was used?
- Which analysis mode was selected?
- Was the prompt aligned with the mode?
- Which deterministic logic was applied?
- Which documents were retrieved?
- What limitations apply?
- What artifact was saved?
- Can the result be reproduced later?

When the backend can answer those questions clearly, FTIAS behaves as a reliable engineering-support system rather than a temporary analysis screen.

CHAPTER 7 - Database and Data Model

The database is the persistent memory of FTIAS. It stores the records that define users, flight tests, dataset versions, uploaded data, documents, saved analyses, reports, and FRAT assessments.

FTIAS uses a relational database model because the application depends heavily on relationships and traceability. A flight test is not just a file. It is connected to dataset versions, ingestion sessions, parameters, data points, analysis jobs, documents, reports, and mission-risk artifacts.

The database supports three major goals:

1. **Persistence**
Important records survive page reloads, container restarts, and later user sessions.
2. **Traceability**
Outputs can be linked back to the flight test, dataset version, analysis job, and source documents used to create them.
3. **Integrity**
Foreign keys and backend-controlled operations protect relationships between records.

The frontend displays database-backed information, but the backend is responsible for reading and writing database records safely.

Main Entities

The main database entities represent the core objects used by the application.

A simplified view of the main entities is:

Entity	Purpose
User	Owns flight tests, documents, analyses, and workflow records
FlightTest	Top-level engineering test record
DatasetVersion	Immutable uploaded dataset version under a flight test
IngestionSession	Operational record of a CSV upload
TestParameter	Definition of a telemetry parameter/channel
DataPoint	Time-series measurement value
AnalysisJob	Immutable saved AI/deterministic analysis artifact
Document	Uploaded technical/reference document
DocumentChunk	Searchable chunk of an uploaded document
FRATAssessment	Mission-risk assessment workflow artifact

Each entity has a different role, but their value comes from how they connect.

User

The User entity represents the application user.

The user is important because FTIAS enforces tenancy. A user should only see and modify the records they own or are permitted to access.

A user may own or create:

- flight tests
- documents
- ingestion sessions
- analysis jobs
- FRAT assessments

The backend must enforce ownership checks. The frontend may hide records that do not belong to the user, but security cannot depend only on frontend visibility.

FlightTest

The FlightTest entity is the main container for flight-test work.

A flight test may include:

- metadata such as name, aircraft type, description, and test date
- dataset versions
- ingestion sessions
- data points
- saved analysis jobs
- FRAT assessments
- active dataset pointer

The flight test is the top-level engineering object. Most workflows begin by selecting or creating a flight test.

A flight test should answer:

- What test is being reviewed?
- Which aircraft or configuration is involved?
- Which dataset versions belong to this test?
- Which analyses or reports were generated from this test?

DatasetVersion

The DatasetVersion entity represents an immutable upload version under a flight test.

Each successful CSV upload should create a new dataset version such as:

- v1
- v2
- v3

The key idea is that uploads do not silently overwrite old data.

A dataset version may store:

- flight test ID
- version number
- dataset label
- status
- row count
- data-point count
- source ingestion session
- creation timestamp

Dataset versions are central to reproducibility. Charts, parameter reads, AI analyses, and reports should be scoped to the selected dataset version.

If a user uploads a corrected CSV later, old analyses should still point to the dataset version that was used when they were created.

IngestionSession

The IngestionSession entity records the operational state of an upload.

It answers:

- When was the upload started?
- Which flight test was it for?
- Which dataset version did it create?
- Did processing succeed or fail?
- What error occurred, if any?

- How many rows or data points were processed?

Typical ingestion states may include:

- pending
- processing
- success
- failed

The ingestion session is not the same as the dataset version.

The ingestion session describes the upload operation.

The dataset version describes the resulting immutable dataset.

This distinction is useful because an upload may fail, while a successful upload creates a usable dataset version.

TestParameter

The TestParameter entity represents a telemetry parameter or channel definition.

Examples include:

- Ground Speed
- Calibrated Air Speed
- Altitude
- WOW / Weight on Wheels
- Aileron Deflection
- Pitch Rate
- Body Normal Acceleration
- Mach
- Static Pressure

A test parameter may include:

- name
- unit
- description or metadata
- ownership or scope information

Parameter definitions allow the system to avoid treating every measurement as an isolated unknown column. They help charts, statistics, and deterministic calculators identify what data is available.

DataPoint

The DataPoint entity stores individual time-series measurement values.

A data point is typically linked to:

- flight test
- dataset version
- parameter
- timestamp
- numeric value

Data points are the main source for:

- chart rendering
- parameter statistics
- deterministic calculations
- analysis summaries
- event detection
- anomaly screening

Because flight-test datasets can be large, data-point insertion and retrieval must be efficient and dataset-scoped.

A data point should always be interpreted in relation to its dataset version. The same flight test may have multiple dataset versions containing similar or different parameter values.

AnalysisJob

The AnalysisJob entity stores a saved analysis artifact.

An analysis job is one of the most important records in FTIAS because it preserves the analysis result as an immutable snapshot.

An analysis job may include:

- flight test ID
- dataset version ID
- user prompt

- selected analysis mode
- executed analysis mode
- model metadata
- deterministic output
- narrative text
- retrieved sources snapshot
- parameter statistics snapshot
- prompt-to-mode guard snapshot
- analysis control snapshot
- output hash
- creation timestamp

The purpose of an analysis job is reproducibility.

If the user reopens Analysis Job #42 later, the system should show the saved result from that job, not a new result based on whatever data is currently active.

Document

The Document entity represents an uploaded reference document.

Documents may include:

- standards
- manuals
- technical reports
- flight-test references
- safety guidance
- certification guidance
- company procedures

A document may store:

- title
- filename
- upload user
- document type

- retrieval metadata
- authority type
- revision
- domain tags
- capability tags

Documents support AI Standards Query and RAG-assisted analysis.

The document itself is a source container. The retrieval system usually works at the chunk level.

DocumentChunk

The DocumentChunk entity represents a searchable section of a document.

Document chunks allow the retrieval system to find relevant passages instead of searching an entire document as one large block.

A document chunk may include:

- document ID
- chunk text
- page or section information
- embedding or search metadata
- similarity score during retrieval
- source metadata

Document chunks support citations and retrieved-source snapshots.

When an AI analysis uses sources, the saved analysis job should preserve the retrieved-source snapshot so the report can explain which references supported the narrative.

FRATAssessment

The FRATAssessment entity represents a FRAT / mission-risk assessment.

A FRAT assessment may include:

- flight test ID
- dataset version ID
- linked analysis job IDs
- input risk factors

- category scores
- total score
- risk band
- hard-stop flags
- lifecycle state
- approval or rejection metadata
- finalization snapshot
- export metadata

The FRAT record is different from a normal AI analysis because it is a workflow artifact with state transitions.

A FRAT may move through states such as:

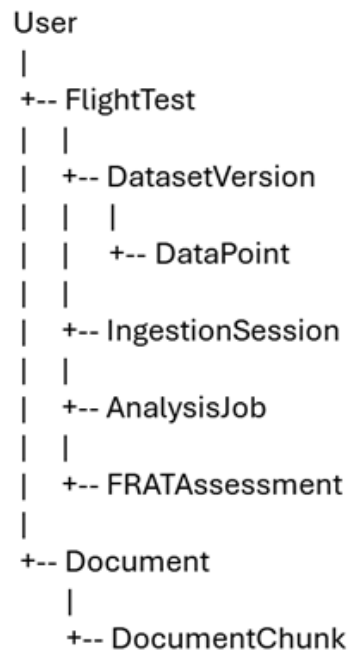
- draft
- scored
- needs review
- approved
- rejected
- finalized

A finalized FRAT should be treated as immutable.

Key Relationships

The database model is best understood as a graph of connected records.

A simplified relationship map is:



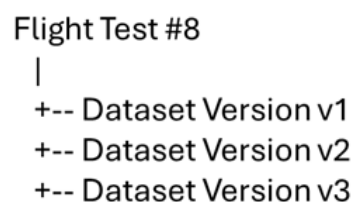
This simplified diagram does not show every foreign key, but it shows the major ownership and workflow relationships.

Flight Test to Dataset Version

A flight test may have many dataset versions.

This relationship supports historical data preservation.

Example:



The active dataset pointer identifies the default dataset version for normal work, but historical versions remain available.

Dataset Version to Data Points

A dataset version contains the data points created during that upload.

This relationship is critical for dataset-scoped reads.

When the user selects dataset version v3, the backend should return data points from v3, not from v1 or v2.

This ensures that charts, statistics, deterministic calculations, and AI analyses use the intended dataset.

Ingestion Session to Dataset Version

An ingestion session records the upload operation that produced a dataset version.

This relationship helps answer:

- Which upload created this dataset?
- Did the upload complete successfully?
- How many rows were processed?
- Did any error occur?

The ingestion session is operational provenance.

The dataset version is analytical provenance.

Both matter.

Flight Test to Analysis Job

A flight test may have many saved analysis jobs.

Each analysis job should also point to the dataset version used when the analysis was created.

This relationship allows the user to reopen previous analyses and understand exactly which data produced them.

Example:

```
Flight Test #8
|
+-- Dataset Version v3
|
+-- Analysis Job #42
    |
    +-- generated against Dataset Version v3
```

The analysis job should not drift when the active dataset changes.

Flight Test to FRAT Assessment

A flight test may have one or more FRAT assessments.

A FRAT assessment may optionally reference dataset versions and supporting analysis jobs.

This allows the mission-risk workflow to consider available analysis evidence while preserving its own independent lifecycle and finalization state.

The FRAT should not be only a frontend form. It must be persisted as a backend-controlled artifact.

Document to Document Chunk

A document may have many chunks.

This relationship supports retrieval.

When the user asks a standards question or runs RAG-assisted analysis, the backend retrieves relevant chunks and uses them as context.

The saved analysis job should preserve the retrieved-source snapshot, because document retrieval results may change if documents are later added, removed, or reprocessed.

Provenance Model

Provenance means the ability to trace where an output came from.

In FTIAS, provenance should answer questions such as:

- Which user created the record?
- Which flight test was used?
- Which dataset version was selected?
- Which ingestion session created the dataset?
- Which parameters were available?
- Which analysis mode was selected?
- Which deterministic calculator was used?
- Which documents or chunks were retrieved?
- Which analysis job was saved?
- Which report was exported?

Provenance is not just documentation. It is a database and backend design principle.

The system should preserve provenance in records, not only in visual page labels.

Dataset Provenance

Dataset provenance connects uploaded data to the flight test and ingestion session that created it.

A dataset version should preserve:

- flight test ID
- version number
- label
- status
- row count
- data-point count
- source ingestion session
- creation timestamp

This allows the user to distinguish between different uploads.

For example, if v3 has 5,640,094 data points, the system should preserve that count and show it consistently in dataset version panels, upload history, analyses, and reports.

Analysis Provenance

Analysis provenance connects a saved analysis to the data and context used to create it.

An analysis job should preserve:

- flight test
- dataset version
- selected mode
- executed mode
- prompt
- deterministic result
- retrieved sources
- parameter statistics
- controls and warnings
- model metadata
- output hash

This allows the user to reopen the job and export a report later.

The analysis should not silently update when the user uploads a new dataset version or selects a different active dataset.

Report Provenance

Report provenance means that a PDF report must clearly state where the report content came from.

A good report should show:

- flight test metadata
- dataset version ID and label
- analysis job ID
- analysis creation time
- model metadata
- parameter snapshot
- retrieved sources
- applicability boundaries
- export time

The report should be generated from the saved analysis job or finalized workflow artifact, not from temporary frontend screen text.

FRAT Provenance

FRAT provenance connects a mission-risk assessment to the relevant operational context.

A FRAT record may include:

- flight test
- dataset version
- supporting analysis jobs
- input risk factors
- scoring results
- hard-stops
- approval or rejection metadata
- finalization timestamp
- exported artifact

A finalized FRAT should be treated as an auditable risk record.

Immutability Model

Immutability means that certain records should not change once they are created or finalized.

FTIAS uses immutability to protect reproducibility.

Important, immutable, or mostly immutable artifacts include:

- dataset versions
- saved analysis jobs
- retrieved-source snapshots
- parameter statistics snapshots
- finalized FRAT records
- exported report artifacts

Immutability does not mean that the database never changes. It means that important analytical artifacts are preserved rather than silently overwritten.

Dataset Immutability

Each successful upload creates a dataset version.

If the user uploads another CSV, the system should create a new dataset version rather than replacing the old one.

This protects earlier analyses and reports.

Example:

Upload 1 -> Dataset Version v1

Upload 2 -> Dataset Version v2

Upload 3 -> Dataset Version v3

If Analysis Job #20 was created from v1, it should remain tied to v1 even after v3 becomes active.

Analysis Job Immutability

An analysis job should preserve the result generated at the time of analysis.

This includes:

- deterministic calculations
- narrative text
- source retrieval snapshot

- controls
- prompt guard result
- dataset version
- model metadata

If a user reopens an old analysis, the system should show the saved job's original provenance.

This is important because later changes may affect:

- active dataset version
- available documents
- retrieval ranking
- frontend labels
- analysis modes
- model behavior

Saved analysis artifacts protect against this drift.

FRAT Immutability

A FRAT assessment may be editable while it is a draft.

However, once finalized, it should become immutable or at least protected from normal editing.

This protects mission-risk review integrity.

A finalized FRAT should preserve:

- input factors
- scores
- hard-stops
- reviewer action
- final decision
- export snapshot

If a later FRAT is needed, create a new assessment rather than silently modifying the finalized record.

Migration Discipline

Database migrations are schema changes applied to PostgreSQL.

Migrations are required when the backend code expects new:

- tables
- columns
- indexes
- constraints
- relationships

Migration discipline is critical because backend code and database schema must match.

If code expects a column that does not exist, the frontend may show a generic error, such as:

Failed to fetch

But the backend logs may show the real cause:

column documents.authority_type does not exist

This means the code has been updated, but the database migration has not been applied.

Safe Migration Practice

A safe migration practice is:

1. pull or receive new code
2. Check whether new migration files were added
3. Apply migrations to PostgreSQL
4. Restart or rebuild the backend
5. Check backend logs
6. Test the affected feature

A common PowerShell command pattern is:

- `Get-Content backend/migrations/<migration_file>.sql -Raw | docker compose exec -T postgres sh -lc 'psql -v ON_ERROR_STOP=1 -U "$POSTGRES_USER" -d "$POSTGRES_DB"'`

After applying the migration:

- `docker compose restart backend`

If frontend contracts also changed:

- `docker compose up -d --build backend frontend`

Foreign Keys and Deletion Order

Foreign keys protect database relationships.

For example, a dataset version may reference an ingestion session. A flight test may point to an active dataset version. An analysis job may reference a dataset version.

If the backend tries to delete a parent record while child records still reference it, PostgreSQL may reject the operation.

This is called a foreign-key violation.

Safe deletion requires deleting or updating records in the correct order.

A simplified delete order for a flight test is:

1. Verify user ownership
2. clear active dataset pointer
3. Delete FRAT assessments
4. delete analysis jobs
5. delete data points
6. delete dataset versions
7. Delete ingestion sessions
8. Delete the flight test

This order may change as the data model grows, but the principle remains:

Remove dependent records before removing the record they depend on.

Database and Data Model Summary

The FTIAS database is not only a storage. It is the backbone of traceability and integrity.

The main ideas are:

- users' own records
- flight tests contain datasets and analyses
- uploads create ingestion sessions and dataset versions
- dataset versions preserve immutable telemetry snapshots
- data points belong to specific dataset versions
- analysis jobs preserve immutable analysis artifacts
- documents and chunks support retrieval
- FRAT records preserve mission-risk workflow state
- reports should export from saved artifacts

- migrations keep the schema aligned with code
- foreign keys protect relationships

A healthy FTIAS data model should always be able to answer:

- What object does this record belong to?
- Which dataset version was used?
- Which upload created the dataset?
- Which analysis job produced the report?
- Which documents supported the answer?
- Which artifact was finalized or exported?

When the database can answer these questions, FTIAS remains traceable, reproducible, and maintainable.

CHAPTER 8 - Data Ingestion Workflow

Data ingestion is the process of taking a user-uploaded CSV file and converting it into structured, database-backed flight-test data.

In FTIAS, ingestion is more than simply importing a file. A successful ingestion creates traceable records that connect the uploaded file, the flight test, the dataset version, the parameters, and the time-series data points.

The ingestion workflow is important because it is the entry point for most engineering analysis. If ingestion is wrong, then charts, deterministic calculations, AI analysis, reports, and FRAT evidence may also be wrong.

For that reason, ingestion must be:

- traceable
- dataset-versioned
- validated
- failure-aware
- tied to user ownership
- protected by backend logic

The frontend provides the upload interface, but the backend controls the ingestion process.

Flight Test Creation

Before data can be uploaded, the user normally creates or selects a flight test.

A flight test is the top-level container for the data and analysis work. It represents the engineering test record, not only the CSV file.

A flight test may include:

- flight-test name
- aircraft type
- test date
- description
- uploaded dataset versions
- parameter data
- saved analysis jobs
- reports

- FRAT assessments

The user creates the flight test from the frontend. The backend then validates the request and creates the database record.

Typical backend checks include:

- user is authenticated
- required fields are present
- the flight-test name is acceptable
- duplicate naming rules are respected
- the new record is linked to the current user

The key principle is ownership. The backend must ensure that users can only upload data into flight tests they own or are allowed to access.

CSV Upload

CSV upload is the user-facing beginning of the ingestion process.

The user selects a flight test, chooses a CSV file, and starts the upload from the frontend.

The frontend sends the file to the backend. The backend then becomes responsible for validating and processing the file.

A CSV file usually contains:

- timestamp column
- multiple telemetry parameter columns
- numeric values
- optional units or metadata depending on the source format

The backend must convert this file into structured records.

A simplified upload sequence is:

1. user selects a flight test
2. user selects a CSV file
3. frontend sends the file to the backend
4. backend confirms flight-test ownership
5. backend creates an ingestion session
6. backend creates a dataset version
7. backend parses the CSV

8. backend creates or reuses parameter definitions
9. backend inserts data points
10. backend updates ingestion and dataset status
11. frontend displays the final result

The upload should not silently overwrite previous data. Each successful upload should become a new dataset version.

Ingestion Sessions

An ingestion session is a database record that tracks the upload operation.

It is not the same as the dataset version.

The ingestion session describes the process.

The dataset version describes the resulting dataset.

An ingestion session helps answer:

- Which upload was attempted?
- Which flight test was involved?
- When did processing start?
- Is the upload pending, processing, successful, or failed?
- Which dataset version did it create?
- How many rows were processed?
- What error occurred if the upload failed?

Typical ingestion states may include:

- pending
- processing
- success
- failed

The ingestion session is important because CSV processing can take time, especially for large datasets.

The frontend can use ingestion status to show progress or history. The backend uses the ingestion session to preserve operational traceability.

Dataset Version Creation

When an upload begins, the backend creates a new dataset version for the selected flight test.

The dataset version is the immutable data container created from the upload.

Example:

- First upload -> Dataset Version v1
- Second upload -> Dataset Version v2
- Third upload -> Dataset Version v3

This design prevents hidden overwrite behavior.

If a user uploads a corrected file, the old dataset remains available. This allows:

- comparison between versions
- reopening old analyses
- reproducible reports
- audit of what data was used

A dataset version should store information such as:

- flight test ID
- version number
- label
- status
- row count
- data-point count
- source ingestion session
- creation timestamp

On success, the new dataset version may become the active dataset version for the flight test.

On failure, the dataset version should be marked failed rather than pretending the data is usable.

Parameter Detection

After the backend opens the CSV, it must identify the timestamp column and telemetry parameter columns.

A telemetry parameter is a measured or derived signal, such as:

- Ground Speed
- Calibrated Air Speed

- Altitude
- Pitch Rate
- Body Normal Acceleration
- Aileron Deflection
- Weight on Wheels
- Mach
- Static Pressure
- Fuel Flow

The backend creates or reuses parameter definitions for the CSV columns.

Parameter definitions are important because the same parameter may be used later by:

- charts
- saved parameter sets
- deterministic calculators
- AI analysis
- report statistics
- FRAT evidence

The backend should preserve parameter names and units where available.

If units are not available or are unclear, the system should avoid making unsupported assumptions.

Timestamp Handling

Timestamp handling is one of the most important validation steps.

Flight-test data is time-series data. Without valid timestamps, the system cannot reliably:

- plot time histories
- compare parameters
- detect events
- estimate takeoff or landing segments
- calculate rates or lags
- identify anomaly windows
- align control inputs with aircraft response

The backend should validate that timestamps are present and parseable.

If timestamp validation fails, the ingestion should fail clearly.

A failure is better than silently importing data with broken timing.

For deterministic analysis, timestamp quality affects many results. For example:

- takeoff distance depends on integrating speed over time
- landing rollout depends on touchdown and deceleration timing
- handling/control-response lag depends on synchronized samples
- vibration and flutter screening depend on sample cadence and event windows

Data Point Insertion

After validating the CSV structure, the backend inserts the telemetry values as data points.

A data point usually includes:

- flight test ID
- dataset version ID
- parameter ID
- timestamp
- value

The dataset version ID is essential. It ensures that each value belongs to the correct upload version.

Because large flight-test files may contain millions of data points, insertion should be efficient and batched where possible.

The backend should update row counts and data-point counts so the frontend and reports can show meaningful dataset information.

Example:

Dataset Version v3

Rows: 60001

Data Points: 5640094

These counts help the user confirm that the upload produced the expected amount of data.

Validation

Validation protects the quality of the ingested dataset.

Typical validation areas include:

- file type
- file readability
- timestamp column presence
- timestamp parseability
- row count
- parameter column detection
- numeric value conversion
- required ownership checks
- dataset version creation
- database insertion success

Validation does not guarantee that the flight-test data is physically correct. It only confirms that the file can be imported into the system in a structured way.

Engineering validation still requires the user to inspect signals, charts, parameter statistics, and analysis results.

For example, ingestion may succeed even if:

- A sensor is noisy
- A parameter unit is wrong
- a channel is mislabeled
- The test data contains unusual values
- the aircraft configuration is not known

The ingestion process should therefore be understood as data import validation, not full engineering validation.

Failure Cases

Ingestion can fail for several reasons.

Common failure cases include:

- missing timestamp column
- invalid timestamp format
- unreadable CSV file

- unsupported file type
- empty file
- nonnumeric values where numeric data is expected
- database connection error
- duplicate or malformed headers
- backend timeout or memory issue
- migration mismatch affecting expected tables or columns

When ingestion fails, the backend should:

1. Mark the ingestion session as failed
2. Mark the dataset version as failed if one was created
3. store an error message or failure reason
4. Avoid treating the dataset as usable
5. Return a clear error response to the frontend

The frontend should clearly display the failure, but the detailed cause is often found in backend logs.

Upload History

Upload History shows the operational record of ingestion attempts.

It may include:

- file name
- upload time
- status
- row count
- dataset version label
- failure reason

Upload History is useful because it shows what happened during data import.

However, Upload History should not be confused with Dataset Versions.

- Upload History = the upload operation record
- Dataset Versions = the usable, immutable datasets created by successful uploads

A failed upload may appear in Upload History, but should not be treated as a successful dataset version.

A successful upload should show a dataset version label that matches the real dataset version, such as v1, v2, or v3.

Active Dataset After Upload

After a successful upload, FTIAS may set the new dataset version as the active dataset for the flight test.

The active dataset is the default version used when the user has not explicitly selected another.

This is convenient because the newest upload is often the dataset the user wants to inspect next.

However, older dataset versions remain available.

The user should always check which dataset version is selected before running an analysis or exporting a report.

This is especially important when comparing results across multiple uploads.

Large Dataset Considerations

Flight-test datasets can be large.

A dataset may contain:

- many parameters
- thousands or millions of data points
- long test durations
- high sample rates
- repeated or redundant channels

Large datasets require careful handling.

The backend should process data efficiently, and the frontend should avoid rendering too much data at once.

Large dataset concerns include:

- upload time
- parsing time
- database insertion performance
- memory usage
- parameter list size
- chart rendering performance
- analysis runtime

FTIAS addresses this through design choices such as:

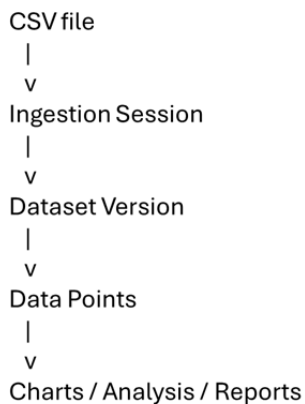
- backend-controlled ingestion
- batched insertion
- dataset-scoped reads
- parameter statistics snapshots
- limited chart overlays
- saved parameter sets
- analysis artifacts

The user should be patient with large uploads and check backend logs if the frontend appears stuck or returns an error.

Ingestion and Provenance

Ingestion is the beginning of the provenance chain.

A successful upload creates a traceable path:



This path allows later outputs to explain the data's source.

For example, a PDF report should be able to state:

- flight test ID
- dataset version ID
- dataset label
- source ingestion session
- number of rows
- number of data points

That provenance is only possible if ingestion records are saved correctly.

User Checklist for Data Ingestion

Before upload:

1. Confirm the correct flight test is selected.
2. Confirm the CSV file is the intended file.
3. Confirm the file contains a timestamp column.
4. Confirm important parameters are included.
5. Confirm the file size is reasonable for the environment.

After upload:

1. Check that the ingestion status is successful.
2. Check that a new dataset version was created.
3. Confirm the dataset label, such as v1, v2, or v3.
4. Check row count and data-point count.
5. Open Parameters and confirm expected channels are available.
6. Plot a few key channels to verify signal quality.
7. Confirm the correct dataset version is selected before analysis.

If upload fails:

1. Read the frontend error message.
2. Check backend logs.
3. Confirm migrations are applied.
4. Confirm the CSV timestamp format is valid.
5. Confirm the file is readable and not empty.
6. Retry only after correcting the suspected cause.

Data Ingestion Workflow Summary

Data ingestion converts a CSV file into structured flight-test data.

The key outputs of ingestion are:

- ingestion session
- dataset version

- parameter definitions
- data points
- row and data-point counts
- status and failure details

The most important ingestion principles are:

- Every successful upload creates a new dataset version
- old datasets should not be silently overwritten
- timestamps must be valid
- data points must be linked to the correct dataset version
- failures must be recorded clearly
- upload history and dataset versions must remain understandable
- analysis and reports depend on correct ingestion provenance

When ingestion is implemented correctly, FTIAS can trace later charts, analyses, reports, and FRAT evidence back to the original uploaded dataset.

CHAPTER 9 - Dataset Versioning and Provenance

Dataset versioning and provenance are core design principles of FTIAS. They ensure that all analysis, charts, and reports can be traced back to the exact data used when they were created.

Without versioning and provenance, engineering outputs lose reliability. With them, the system becomes reproducible and auditable.

Dataset Versioning Concept

In FTIAS, every successful CSV upload creates a new immutable dataset version.

Example:

- Upload 1 → Dataset Version v1
- Upload 2 → Dataset Version v2
- Upload 3 → Dataset Version v3

No dataset is overwritten. Each version represents a snapshot of the data at the time of upload.

This allows:

- comparison between datasets
 - rollback to previous data
 - reproducible analysis
 - audit of data evolution
-

Active Dataset

Each flight test may have multiple dataset versions, but only one is marked as active.

The active dataset is:

- the default dataset used in charts
- the default dataset used in AI analysis
- the default dataset shown in the UI

However, users can manually select other versions.

Important rule:

- **The active dataset ≠ is the only dataset available**
-

Dataset Selection in Workflows

Dataset version selection affects:

- parameter lists
- chart rendering
- deterministic calculations
- AI analysis
- report generation

If the wrong dataset version is selected, results may be misleading.

The frontend must clearly show:

- available versions
 - selected version
 - active version
-

Dataset Provenance

Dataset provenance answers:

- Where did this data come from?
- Which upload created it?
- How many data points exist?
- When was it created?

Each dataset version should preserve:

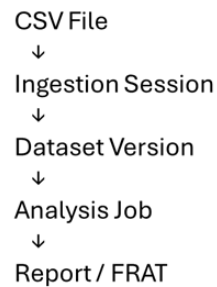
- flight test ID
- version label (v1, v2, ...)
- ingestion session reference
- row count
- data-point count
- creation timestamp

Example:

- Dataset Version v3
- Rows: 60001
- Data Points: 5640094
- Source: Ingestion Session #27

Provenance Chain

Dataset versioning forms the base of a larger provenance chain:



Every step should preserve traceability.

Why Versioning Matters

Without versioning:

- analyses could change silently
- Reports could become inconsistent
- debugging becomes difficult
- engineering trust is reduced

With versioning:

- results are reproducible
 - changes are explicit
 - historical comparison is possible
-

Dataset Versioning Summary

Key principles:

- uploads create new versions
- versions are immutable
- active dataset is only a pointer
- analysis must reference specific versions
- provenance must be preserved

CHAPTER 10 - AI Analysis Workflow

The AI Analysis workflow is the core analytical engine of FTIAS. It combines deterministic engineering logic, dataset statistics, document retrieval, and structured AI generation.

The goal is not just to generate text, but to produce **traceable, bounded engineering insight**.

Analysis Workflow Overview

A typical analysis flow is:

1. user selects flight test
 2. user selects dataset version
 3. user selects analysis mode
 4. user writes prompt
 5. frontend sends request
 6. backend validates request
 7. backend applies prompt-to-mode guard
 8. backend runs deterministic calculations
 9. backend retrieves document context (RAG)
 10. backend builds analysis response
 11. backend stores analysis job
 12. frontend displays results
-

Analysis Modes

FTIAS uses **mode-driven analysis**.

Modes define:

- which deterministic logic applies
- which parameters are expected
- what outputs are valid
- applicability boundaries

Current modes include:

- Takeoff Performance
- Landing Performance

- Performance / Air Data
 - Handling / Control Response
 - Vibration & Loads
 - Flutter Support Pre-screen
 - General Summary
-

Deterministic vs AI Components

Each analysis combines:

Deterministic Components

- calculated metrics
- signal-based detection
- engineering logic

AI Components

- narrative explanation
- interpretation
- recommendations
- standards context

Rule:

Deterministic outputs have higher authority within defined limits.

Prompt-to-Mode Guard

The prompt-to-mode guard ensures alignment between:

- selected mode
- user intent

Example:

Prompt:

“Analyze aileron deflection and stick position.”

Selected mode:

Takeoff

Result:

- mismatch detected
- suggested mode: Handling
- warning displayed

The system may:

- warn only
- downgrade execution
- adjust routing

Capability Routing

The backend uses the **capability catalog** to decide:

- whether a mode is implemented
- whether the required data exists
- whether to allow, limit, or block execution

Possible outcomes:

- allow
- allow with limitations
- downgrade
- reject

Analysis Controls

Each analysis includes structured control fields:

- deterministic confidence
- retrieval coverage
- applicability status
- warning level
- result strength
- downgrade reason

These prevent misinterpretation.

RAG (Retrieval-Augmented Generation)

RAG provides:

- document-based context
- standards references
- cross-check support

It uses:

- document chunks
- metadata
- mode-aware filtering

Important:

RAG supports analysis but does not replace deterministic results.

Saved Analysis Job

Each analysis is saved as an immutable artifact.

It includes:

- dataset version
- prompt
- mode
- deterministic results
- retrieved sources
- controls
- narrative
- timestamps

This allows:

- reopening by ID
 - reproducible reports
 - audit trace
-

Reopen Analysis

When reopening:

- original dataset is preserved
- original results are shown
- The current dataset does not overwrite it

This is critical for traceability.

Analysis Limitations

FTIAS analysis is:

- engineering support
- not a certification authority
- bounded by available data
- limited by implemented models

Examples:

- takeoff $\neq$ certified takeoff distance
 - flutter support $\neq$ flutter clearance
 - vibration $\neq$ structural substantiation
-

AI Analysis Summary

Key principles:

- mode-driven
- deterministic-first
- RAG-supported
- guarded by controls
- saved as immutable artifacts

CHAPTER 11 - Charts, Parameters, and Comparison Features

Charts and parameter exploration are the primary tools for understanding raw flight-test data before analysis.

They allow the user to validate signals, detect anomalies, and build confidence in the dataset.

Parameter Explorer

The Parameter Explorer allows users to:

- search parameters
- select parameters
- view metadata
- organize large datasets

This is critical when datasets contain hundreds of channels.

Parameter Selection

Users can:

- select multiple parameters
- combine signals in charts
- analyze relationships

Example:

- speed vs time
 - control vs response
 - acceleration vs altitude
-

Favourites

Users can mark parameters as favourites.

This helps:

- reduce search time
- focus on key signals
- standardize workflows

Saved Parameter Sets

Users can save parameter groups.

Example:

- takeoff set
- handling set
- vibration set

Benefits:

- reuse across sessions
- faster workflow
- consistent analysis

Charting

Charts allow:

- time-series visualization
- overlay of multiple parameters
- zoom and inspection
- trend identification

Charts are essential before running an analysis.

Overlays and Markers

Charts may include:

- thresholds
- event markers
- annotations

These help identify:

- liftoff
- touchdown
- anomalies

- transitions
-

Dataset Comparison

Users can compare:

- multiple dataset versions
- same parameter across versions

Example:

v1 vs v3 ground speed

This helps detect:

- corrections
 - improvements
 - inconsistencies
-

Chart Export

Users can export charts as images.

This supports:

- reporting
 - presentations
 - documentation
-

Role of Charts in Analysis

Charts are not just visualization tools.

They are:

- validation tools
- debugging tools
- engineering evidence

Before running the analysis, users should:

- inspect key signals
- confirm data quality

- verify parameter availability
-

Common Issues

Missing parameters

Possible causes:

- wrong dataset selected
 - ingestion issue
 - parameter naming mismatch
-

Incorrect charts

Possible causes:

- unit mismatch
 - noisy data
 - timestamp issues
-

Performance issues

Possible causes:

- too many parameters selected
 - large dataset
 - rendering limitations
-

Best Practices

Before analysis:

1. Check the dataset version
 2. Inspect key parameters
 3. Verify signal quality
 4. Confirm required channels exist
-

Charts and Parameters Summary

Key principles:

- charts support understanding
- parameters define data scope
- datasets must be verified visually
- comparison enables validation
- analysis should follow chart inspection

CHAPTER 12 - Report and PDF Generation

The report generation workflow in FTIAS transforms a saved analysis artifact into a structured, reproducible engineering document. Unlike temporary screen output, reports are built from immutable data and preserve the analysis context.

Purpose of Report Generation

The main purpose of report generation is to:

- provide a **traceable engineering output**
- preserve **analysis provenance**
- communicate **results, limitations, and controls clearly**
- enable **review, sharing, and documentation**

A report is not simply a formatted version of the screen. It is a reconstructed artifact based on saved analysis data.

Immutable Artifact Principle

Reports must be generated from:

- saved analysis jobs
- stored deterministic outputs
- saved control fields
- preserved dataset provenance

They must **not depend on the current frontend state.**

This ensures:

- reproducibility
- auditability
- consistency over time

Example:

- Analysis created on dataset v1
 - Later dataset v3 becomes active
 - Report must still reflect **v1**, not v3
-

Report Generation Workflow

1. User runs or reopens an analysis
 2. The analysis job is retrieved from the backend
 3. User requests report export
 4. Backend loads saved artifact
 5. Backend composes report sections
 6. PDF is generated and returned
-

Report Structure

A typical report includes:

- Flight-test metadata
 - Dataset version and provenance
 - Analysis mode and prompt
 - Deterministic results
 - Parameter statistics snapshot
 - Retrieved source references
 - Analysis narrative
 - Controls and warnings
 - Applicability boundaries
 - Assumptions and limitations
 - Provenance statement
-

Controls in Reports

Reports include structured control fields:

- deterministic confidence
- retrieval coverage
- applicability status
- warning level
- result strength
- downgrade reason

These fields are critical because they prevent misinterpretation.

Example:

- High confidence but **limited applicability**
 - Strong warning level in flutter screening
 - Advisory-only result
-

Warning Escalation

Warnings should be clearly visible in reports.

Typical escalation cases:

- insufficient data quality
- missing required parameters
- screening-only mode used
- strong mismatch between prompt and mode
- downgraded execution

The report must make clear:

“This result is limited and should not be used as standalone evidence.”

Provenance in Reports

Each report should answer:

- Which flight test was used?
- Which dataset version was used?
- Which analysis mode was executed?
- What deterministic logic was applied?
- What sources were retrieved?
- When was the analysis created?

This ensures **engineering traceability**.

Limitations of Reports

Reports are:

- engineering support artifacts
- not certification documents
- dependent on input data quality
- bounded by implemented models

They must not be interpreted as:

- automatic approval
- certification evidence
- final authority

Report Generation Summary

Key principles:

- reports are generated from saved artifacts
- provenance must be explicit
- controls must be visible
- warnings must be clear
- results must remain bounded

A correct report is not only informative — it is **defensible**.

CHAPTER 13 - FRAT / Mission Risk Workflow (Detailed)

The FRAT (Flight Risk Assessment Tool) workflow provides a structured method to evaluate mission or test risk using predefined factors, scoring logic, and controlled lifecycle states.

Purpose of FRAT

FRAT is used to:

- evaluate mission or test risk
- structure decision-making
- identify high-risk conditions
- enforce review through hard-stop logic
- document approval decisions

It is not a replacement for operational authority. It is a **decision-support tool**.

FRAT Workflow Overview

1. Create draft assessment
 2. Enter risk factors
 3. Compute score
 4. Evaluate hard stops
 5. Review assessment
 6. Approve or reject
 7. Finalize
 8. Export
-

Risk Scoring

FRAT scoring is deterministic.

It evaluates:

- environmental conditions
- aircraft condition
- crew readiness

- mission complexity
- operational constraints

The system produces:

- category scores
- total score
- risk classification

Hard-Stop Logic

Hard stops override normal scoring.

Examples:

- critical system issue
- extreme environmental condition
- missing required approval
- severe anomaly in analysis

If a hard stop is triggered:

- The assessment cannot proceed normally
- additional review is required

Lifecycle States

FRAT assessments move through states:

- draft
- scored
- under review
- approved
- rejected
- finalized

Only valid transitions are allowed.

Example:

- finalized → cannot return to draft

User Checklist for FRAT

Before scoring:

- Are all risk inputs complete?
- Is the correct dataset or context referenced?

Before approval:

- Are hard stops resolved or accepted?
- Are risks understood and documented?
- Is supporting analysis available?
- Are warning levels acceptable?

Before finalization:

- Is the decision intentional?
- Is the review complete?

Interpretation of FRAT Results

FRAT output must be interpreted carefully:

- A low score does not guarantee safety
- A high score does not automatically prohibit operation
- hard stops must always be respected

FRAT supports judgment — it does not replace it.

FRAT and Analysis Integration

FRAT may reference:

- analysis jobs
- dataset versions
- reports

This allows risk decisions to be supported by technical evidence.

FRAT Summary

Key principles:

- structured risk evaluation
- deterministic scoring
- hard-stop enforcement
- controlled lifecycle
- immutable final artifact

CHAPTER 14 - Analysis Interpretation Guide

FTIAS produces structured analysis results, but correct interpretation depends on understanding the system's controls, limitations, and data quality.

Reading an Analysis Result

Each analysis includes:

- deterministic outputs
- narrative explanation
- control fields
- retrieved sources
- warnings and limitations

The user must interpret all of these together.

Understanding Control Fields

Key fields:

- deterministic confidence → how reliable the calculation is
 - retrieval coverage → how strong the document support is
 - applicability status → where the result applies
 - warning level → severity of concerns
 - result strength → overall interpretation level
-

Data Quality Impact

Analysis quality depends on input data.

Examples:

- missing parameters → blocked or partial result
 - noisy signals → reduced confidence
 - bad timestamps → invalid timing-based calculations
 - incomplete dataset → limited applicability
-

Mode Limitations

Each mode has boundaries:

- Takeoff → not certification distance
- Handling → not formal HQ rating
- Vibration → screening only
- Flutter → pre-screen only

Results must remain within these limits.

When to Trust Results

Stronger results:

- required parameters present
 - clear signal quality
 - deterministic output available
 - high confidence
 - low warning level
-

When to Be Cautious

Be cautious when:

- warnings are high
 - applicability is partial
 - result is advisory
 - deterministic output is missing
 - prompt-mode mismatch occurred
-

When to Escalate

Escalate to deeper analysis when:

- Flutter concerns detected
- vibration anomalies persist
- inconsistent performance results

- unclear control-response behavior
-

Interpretation Summary

- analysis is structured, not absolute
- controls define interpretation limits
- data quality directly affects results
- user judgment remains essential

CHAPTER 15 - System Limitations and User Responsibility

FTIAS is an engineering support system. It does not replace certified analysis tools, formal verification processes, or qualified engineering judgment.

System Limitations

FTIAS is limited by:

- available telemetry data
 - implemented deterministic models
 - signal quality
 - absence of certification corrections
 - bounded analysis modes
-

Not a Certification Tool

FTIAS does not provide:

- certified performance values
 - approved safety determinations
 - regulatory compliance validation
-

User Responsibility

The user is responsible for:

- selecting the correct dataset
 - verifying data quality
 - choosing an appropriate analysis mode
 - reviewing warnings and controls
 - interpreting results correctly
 - validating conclusions
-

Responsible Usage

FTIAS should be used for:

- exploratory analysis
 - engineering insight
 - trend identification
 - preliminary evaluation
 - report drafting
-

Responsibility Summary

FTIAS assists the engineer.

It does not replace the engineer.

CHAPTER 16 - Troubleshooting

Failed to Fetch

Usually indicates backend failure.

Check:

- backend running
 - logs
 - migrations
-

Migration Errors

Symptoms:

- missing columns
- API failures

Fix:

- apply migrations
 - restart backend
-

Dataset Issues

- wrong version selected
 - ingestion failed
 - inconsistent labels
-

Analysis Issues

- missing parameters
 - wrong mode
 - prompt mismatch
-

Frontend Issues

- rebuild required
- cache problems

Troubleshooting Summary

Always check:

1. backend logs
2. dataset selection
3. migrations
4. analysis mode

CHAPTER 17 - References and Technology Links

This chapter lists the main technologies, tools, and external concepts used in FTIAS. The purpose is to help maintainers and users understand the project's technical foundation and know where to find official documentation.

Application Architecture

Frontend/Web Application

React

React is used to build the browser-based user interface. It supports reusable components, page routing, state handling, and dynamic rendering of analysis results, charts, forms, and workflow panels.

Reference:

<https://react.dev/>

TypeScript

TypeScript adds static typing to JavaScript and helps reduce errors in frontend contracts, API responses, component props, and application state.

Reference:

<https://www.typescriptlang.org/>

Vite

Vite is commonly used as a fast frontend development and build tool. It supports modern JavaScript/TypeScript workflows and efficient local development.

Reference:

<https://vite.dev/>

pnpm

pnpm is a JavaScript package manager for installing and managing frontend dependencies.

Reference:

<https://pnpm.io/>

Backend/API Layer

Python

Python is the main language used by the backend service.

Reference:

<https://www.python.org/>

FastAPI

FastAPI is used to expose backend REST APIs. It supports request validation, typed schemas, automatic API documentation, and integration with Python backend logic.

Reference:

<https://fastapi.tiangolo.com/>

Pydantic

Pydantic is used for structured data validation and typed request/response models.

Reference:

<https://docs.pydantic.dev/>

SQLAlchemy

SQLAlchemy is used as the backend database ORM / data access layer.

Reference:

<https://www.sqlalchemy.org/>

Pytest

pytest is used for backend automated testing.

Reference:

<https://docs.pytest.org/>

Database and Persistence

PostgreSQL

PostgreSQL is the relational database used to store users, flight tests, dataset versions, ingestion sessions, data points, analysis jobs, documents, document chunks, and FRAT records.

Reference:

<https://www.postgresql.org/>

Database migrations

Database migrations are SQL scripts used to evolve the database schema when new tables or columns are added.

Reference:

<https://www.postgresql.org/docs/>

Runtime and Deployment

Docker

Docker packages application services into containers so they can run consistently across environments.

Reference:

<https://docs.docker.com/>

Docker Compose

Docker Compose starts and connects the frontend, backend, and PostgreSQL services.

Reference:

<https://docs.docker.com/compose/>

AI, Retrieval, and Reporting Concepts

Retrieval-Augmented Generation (RAG)

- RAG combines document retrieval with AI-generated responses. In FTIAS, retrieved documents support standards cross-checking and engineering context, but deterministic calculations remain authoritative where implemented.

Reference:

<https://www.promptingguide.ai/techniques/rag>

OpenAI API / AI model provider

- The backend can use external AI services for narrative generation, standards support, and analysis explanation.

Reference:

<https://platform.openai.com/docs>

PDF reporting

- FTIAS exports reports from saved analysis artifacts so that outputs remain traceable and reproducible.
-

Engineering and Flight-Test Reference Areas

The following areas are relevant to FTIAS engineering workflows:

- flight-test data analysis
- aircraft performance analysis
- takeoff and landing performance

- handling qualities
- air-data and atmosphere calculations
- vibration and loads screening
- flutter-support pre-screening
- safety and risk assessment
- FRAT/mission risk review

Recommended reference categories include:

- flight-test engineering textbooks
 - aircraft performance flight-test guides
 - certification flight-test guides
 - handling qualities references
 - vibration/loads/aeroelasticity references
 - flight-test safety and risk-management references
 - internal company procedures and test plans
-

CHAPTER 18 - Definitions and Key Concepts

This chapter defines the main technical and operational terms used throughout the FTIAS manual.

Active Dataset

The dataset version is currently selected as the default working dataset for a flight test. Other dataset versions may still exist and remain available for historical review or comparison.

AI Analysis

A flight-test-data-centered analysis workflow that combines selected dataset information, deterministic calculations, retrieved document context, and AI-generated narrative. AI analyses are saved as immutable analysis jobs.

AI Standards Query

A document-centered workflow where the user asks questions against uploaded reference documents. It is primarily used to interpret standards and provide technical context.

Analysis Job

A saved, immutable analysis artifact. It stores the prompt, selected mode, dataset version, model metadata, output text, retrieved-source snapshot, parameter statistics snapshot, controls, and provenance.

Analysis Mode

The selected engineering analysis category is used to route a request. Examples include takeoff, landing, performance, handling qualities, vibration and loads, flutter support, and general summary.

Applicability Boundary

The stated limit within which an analysis result may be interpreted. For example, a takeoff ground-roll estimate may be valid for the detected WOW/ground-speed segment but not valid as a certification-corrected takeoff distance.

Backend

The server-side application layer. It validates requests, enforces ownership, reads and writes the database, runs deterministic logic, manages AI/RAG workflows, and generates reports.

Capability Catalog

The backend source of truth that defines supported analysis capabilities, required inputs, authority classification, blocked conditions, and applicability limits.

Confidence/Coverage/Applicability Controls

Structured fields are used to communicate the quality of results. These include deterministic confidence, retrieval coverage, applicability status, warning level, result strength, and the reason for downgrade or blocking.

DataPoint

A stored time-series measurement associated with a flight test, parameter, timestamp, value, and dataset version.

Dataset Version

An immutable upload version under a flight test. Each successful CSV upload creates a new dataset version, such as v1, v2, or v3.

Deterministic Calculation

A calculation performed by explicit programmed logic rather than free-form AI text. Deterministic calculations are preferred for engineering metrics when implemented.

Docker

The container runtime is used to package and run FTIAS services consistently.

Docker Compose

The tool used to start and connect the frontend, backend, and PostgreSQL services.

Document Chunk

A searchable section of an uploaded document used for retrieval and citation support.

Document Library

The frontend area for managing uploaded reference documents.

External AI Service

An external model provider used by the backend for narrative generation, standards interpretation, and RAG-assisted responses.

Flight Test

The top-level engineering record that contains dataset versions, parameter data, analysis jobs, reports, and related workflow artifacts.

Foreign Key

A database relationship that links one record to another. Foreign keys protect data integrity and explain why the order of deletion matters.

FRAT

Flight Risk Assessment Tool. In FTIAS, the FRAT workflow supports structured mission-risk assessment, scoring, hard-stops, approval/rejection, finalization, and export.

Frontend

The browser-based user interface. It displays pages, charts, forms, warnings, analysis results, reports, and workflow controls.

Hard-Stop

A condition that overrides normal scoring or workflow progression. In FRAT, hard stops can force review or prevent approval even if the total score appears acceptable.

Immutable Artifact

A saved record that should not change when later data changes. Examples include saved analysis jobs, PDF reports based on saved jobs, and finalized FRAT records.

Ingestion Session

A persistent record of a CSV upload attempt. It stores operational status such as pending, processing, success, or failure.

Migration

A database schema update script. Migrations must be applied when the backend code expects new tables or columns to be created.

Mode-Aware Retrieval

Retrieval logic that uses the selected analysis mode to prioritize relevant documents and chunks.

Parameter Explorer

The UI tool for searching, selecting, favouriting, and saving groups of parameters for plotting and review.

Prompt-to-Mode Guard

A safety layer that compares the user's written prompt against the selected analysis mode. It warns when the prompt appears to match a different capability.

Provenance

The trace of where an output came from. In FTIAS, provenance includes flight test, dataset version, analysis job, model metadata, retrieved sources, and saved snapshots.

RAG

Retrieval-Augmented Generation. A method where relevant document content is retrieved and supplied as context for AI-generated responses.

Report Export

The process of generating a PDF report from a saved analysis job or workflow artifact.

Result Strength

A control field describing how strongly an output should be interpreted. Examples include authoritative, bounded, advisory, and blocked.

Screening

A bounded engineering review that identifies possible concern areas but does not provide formal substantiation or clearance.

Telemetry

Recorded flight-test measurement data, such as speed, altitude, acceleration, control position, angular rates, pressure, and temperature.

Test Parameter

A parameter definition that stores the name, unit, and metadata for a measured telemetry channel.

Traceability

The ability to explain how an output was produced, including which dataset, analysis mode, calculation, source documents, and saved artifact were used.

WOW

Weight on Wheels—a signal used to determine whether the aircraft is on the ground or airborne.

ANNEX A - Operational Checklists

ANNEX A.1 — Data Upload Checklist

Before upload:

- Correct flight test selected
- CSV contains a valid timestamp
- Required parameters present
- File size acceptable

After upload:

- Status = success
- Dataset version created (vX)
- Row/data-point count verified
- Key parameters visible
- Signals checked in charts

ANNEX A.2 — AI Analysis Checklist

Before running analysis:

- Correct dataset version selected
- Required parameters available
- Appropriate analysis mode selected
- Prompt aligned with mode

After running analysis:

- Check deterministic outputs
- Review control fields (confidence, applicability)
- Review warnings
- Confirm the relevance of the retrieved sources

Before exporting:

- Correct analysis job opened
- Dataset provenance verified

- Limitations understood

ANNEX A.3 — FRAT Checklist

Before scoring:

- All risk inputs completed
- Mission/test context understood

Before approval:

- Hard-stops reviewed
- Supporting analysis considered
- Warning levels acceptable
- Risks clearly understood

Before finalization:

- Decision is intentional
- Review completed
- No unresolved critical issues

ANNEX A.4 — Dataset Version Control Checklist

- Confirm active dataset
- Verify dataset label (v1, v2, v3...)
- Ensure the correct dataset is used for analysis
- Avoid mixing results from different versions
- Reopen analysis jobs to confirm provenance

ANNEX A.5 — Troubleshooting Quick Checklist

If "Failed to fetch":

- Check backend status
- Check logs
- Check migrations

If the analysis looks wrong:

- Check dataset version

- Check parameters
- Check mode selection

If UI is not updated:

- Rebuild frontend
- Clear cache

ANNEX B - FRAT Input Interpretation Guide

This annex provides guidance on evaluating and scoring each FRAT input field. The goal is to ensure consistent, meaningful, and defensible risk assessments.

FRAT scoring is not only numerical. It reflects engineering judgment applied in a structured way.

B.1 General Scoring Philosophy

Each input represents a **risk contributor**.

- Low score → low risk contribution
- High score → increased operational or technical risk

The total score reflects cumulative exposure, but:

Hard stops always override numerical scoring

Users should avoid:

- guessing values
- using default values without evaluation
- compensating one high-risk area by lowering another

Each category must be evaluated independently.

Example Interpretation Bands (organization-dependent):

- 0–25 → Low risk (normally acceptable)
- 26–45 → Moderate risk (review required)
- 46–65 → High risk (mitigation required)
- 66+ → Unacceptable Risk/No-Go unless formally resolved or accepted by defined authority (unacceptable/no_go)

B.2 Assessment Inputs

Assessment Name

Purpose:

- Identify the mission/test clearly

Best practice:

- Use descriptive names (e.g., *High-Speed Taxi Test – Run 3*)

Dataset Version

Purpose:

- Define the data basis for analysis and supporting evidence

Guidance:

- Use the dataset that best represents the current test condition
- Avoid using outdated datasets unless explicitly required

Impact:

- Incorrect dataset selection may invalidate analysis support

Requested Authority

Purpose:

- Define the level of decision authority required

Typical interpretation:

- Low → routine operation
- High → requires senior review or approval

Impact:

- Higher authority implies higher consequence if risk is misjudged

Manual Adjustment (-10 to +10)

Manual adjustment should not be used to override:

- *hard-stop conditions*
- *clearly high-risk categories*

It is intended only for:

- *unmodeled factors*
- *additional uncertainty*
- *justified mitigation not reflected in inputs*

Purpose:

- Allow expert override of computed score

Guidance:

- Use only when justified
- Must be supported with rationale notes

Examples:

- +5 → known uncertainty not captured by inputs
- -5 → mitigating conditions not represented in scoring

Impact:

- Directly increases or decreases total risk score

Linked Analysis Job

Purpose:

- Connect FRAT to technical analysis evidence

Guidance:

- Link relevant analysis when available
- Especially important for:
 - performance evaluation
 - vibration concerns
 - flutter screening

Impact:

- Strengthens the justification of the assessment
- Improves traceability

B.3 Risk Categories (0–20 each)

Each category contributes independently to total risk.

“All categories are equally weighted in scoring, but not necessarily in operational consequence.”

The FRAT scoring table

Score Range	Level	Meaning
0–4	Minimal	Normal condition, no meaningful added risk
5–8	Low	Minor concern, manageable with standard controls
9–12	Moderate	Noticeable risk contributor, requires awareness/monitoring
13–16	High	Significant risk contributor, mitigation/review required
17–20	Critical	Severe contributor, escalation, or no-go unless formally accepted

Mission/Test Profile

Considers:

- complexity of the test
- novelty of conditions
- proximity to operational limits

Examples:

- 0–5 → routine test
- 6–12 → moderate complexity
- 13–20 → high-risk / edge-of-envelope

Weather / Environment

Considers:

- wind conditions
- turbulence
- temperature extremes
- visibility

Examples:

- 0–5 → stable conditions
- 6–12 → moderate variability
- 13–20 → adverse or limiting conditions

Runway / Operational

Considers:

- runway length
- surface condition
- operational constraints
- obstacles or restrictions

Examples:

- 0–5 → normal operations
- 6–12 → constrained environment
- 13–20 → high operational limitation

Aircraft System Status

Considers:

- system availability
- deferred defects
- instrumentation reliability

Examples:

- 0–5 → fully operational
- 6–12 → minor issues
- 13–20 → degraded or uncertain systems

Crew Readiness

Considers:

- crew experience
- fatigue
- familiarity with test profile

Examples:

- 0–5 → fully prepared
- 6–12 → moderate concerns

- 13–20 → significant readiness issues

B.4 Hard-Stop Flags

Hard stops override scoring and require immediate attention.

Critical System Unavailable

Meaning:

- The essential aircraft system is not functional

Impact:

- Operation shall not proceed unless the condition is resolved or formally accepted under the defined authority.

Mandatory Data Missing

Meaning:

- required data for safe execution or analysis is not available

Impact:

- increases uncertainty significantly

Crew Unfit

Meaning:

- crew not physically or mentally prepared

Impact:

- immediate operational risk

Hard-Stop Principle

If a hard stop is triggered, the FRAT must not be treated as “acceptable” based on score alone.

B.5 Notes Sections

Review Notes

Purpose:

- document assessment reasoning

Best practice:

- explain major scoring decisions

Override / Rationale Notes

Required when:

- manual adjustment is used

Must include:

- justification
- reasoning
- supporting context

Transition Notes

Purpose:

- track changes between states

Useful for:

- audit
- review history

B.6 Final Score Interpretation

"The total score provides the risk classification used by the workflow. Organizational rules may define moderate or higher results as No-Go until reviewed, mitigated, or formally accepted."

Guidance:

- Low score → proceed with standard awareness
- Medium score → increased monitoring required
- High score → review and mitigation required

However:

Score alone must never override hard-stop conditions or expert judgment

B.7 Total FRAT Score Composition

The final FRAT score is composed of:

- Final Score =
 - Category Base Score + Manual Adjustment + Analysis Indicator Score

Where:

- Category Base Score =
 - Mission/Test Profile
 - Weather/Environment
 - Runway/Operational
 - Aircraft System Status
 - Crew Readiness
- Manual Adjustment = User-entered adjustment from -10 to +10.
- Analysis Indicator Score = Additional penalty derived from linked analysis controls, capped at 30.

The final score may therefore be higher than the simple sum of the five category scores.

B.8 Key Principles

- Each category must be evaluated independently
- Do not “balance” risks artificially
- Use manual adjustment carefully and transparently
- Always consider hard stops first
- Link analysis when available
- Document reasoning clearly

B.9 Example

Low Risk, Go

- Categories:
 - "Mission Profile": "score": 4, "notes": "Routine mission profile."
 - "Weather Environment": "score": 3, "notes": "Stable weather."
 - "Runway Operational": "score": 2, "notes": "Normal runway status."
 - "Aircraft System Status": "score": 1, "notes": "No major defects."
 - "Crew Readiness": "score": 2, "notes": "Crew ready."
 - "Manual Adjustment": 1,
 - "Critical Flags":
 - "Critical System Unavailable": false,

- "Mandatory Data Missing": false,
 - "Crew Unfit": false
- "Requested Decision Authority": "authoritative"
- Analysis controls:
 - "Warning Level": "none",
 - "Result Strength": "authoritative",
 - "Applicability Status": "fully applicable",
 - "Retrieval Coverage": "strong",
 - "Deterministic Confidence": "high."
- Expected Result:
 - "Base Score": 12,
 - "Manual Adjustment": 1,
 - "Analysis Indicator Score": 0,
 - "Total Score": 13,
 - "Risk Band": "low",
 - "Recommendation": "go",
 - "Hard Stop Triggered": false,
 - "Analysis Controls Used": 1

Hard Stop, No-Go

- Categories:
 - "Mission Profile": "score": 8, "notes": "Complex mission."
 - "Weather Environment": "score": 10, "notes": "Weather concerns."
 - "Runway Operational": "score": 7, "notes": "Runway limitations."
 - "Aircraft System Status": "score": 5, "notes": "Minor aircraft issues."
 - "Crew Readiness": "score": 6, "notes": "Crew workload elevated."
 - "Manual Adjustment": 0,
 - "Critical Flags":
 - "Critical System Unavailable": false,
 - "Mandatory Data Missing": true,

- "Crew Unfit": false
 - "Requested Decision Authority": "authoritative."
- Analysis controls:
 - "Warning Level": "high",
 - "Result Strength": "bounded",
 - "Applicability Status": "partially applicable",
 - "Retrieval Coverage": "weak",
 - "Deterministic Confidence": "low."
- Expected result:
 - "Base Score": 36,
 - "Manual Adjustment": 0,
 - "Analysis Indicator Score": 25,
 - "Total Score": 61,
 - "Risk Band": "unacceptable",
 - "Recommendation": "no-go",
 - "Hard Stop Triggered": true,
 - "Analysis Controls Used": 1
- Expected hard stop includes:
 - "code": "Mandatory Data Missing."
 - "severity": "high"
 - "code": "Analysis High Warning",
 - "severity": "high"

Abbreviations

Acronym	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CAS	Calibrated Airspeed
CSV	Comma-Separated Values
FRAT	Flight Risk Assessment Tool
FTIAS	Flight Test Interactive Analysis Suite
HTTP	Hypertext Transfer Protocol
ID	Identifier
ISA	International Standard Atmosphere
JSON	JavaScript Object Notation
PDF	Portable Document Format
PNG	Portable Network Graphics
RAG	Retrieval-Augmented Generation
SQL	Structured Query Language
TAS	True Airspeed
UI	User Interface
WOW	Weight on Wheels